퀀트투자 시스템 개발을 이어서 하자. 그런데, 기존에 개발하던 전략은 그대로 유지하되 새로운 전략을 따로 좀 개발해야 겠어.
새로운 개발 전략은 아래와 같이 세팅을 하려고 해.
1. 기존의 분류 방법처럼 DART 자료를 기반으로 유니버스의 200개 종목을 뽑아서, 5단계의(1년, 6개월, 3개월, 3주, 3일) 기간 동안의 주가 흐름을 3방향(상승, 횡보, 하락)의 흐름으로 구분해서 총 15개 그룹으로 분류한다.
2. 백테스트를 통해서 유니버스 종목이 각 그룹에서 어느정도(몇 거래일) 머물고, 그다음 어느 그룹으로 이동하는지를 추적해서 하락, 횡보 방향에서 상승빙향으로 들어가는 종목을 찾아낸다. 그룹을 셀로 보고 다른 셀로의 이동 경로를 추적하는 것임.
3. 상승방향으로 들어간 종목들의 RSI, MACD 등 보조지표를 분석해서 매수, 매도, 보유 판단 신호를 생성한다.
이런 전략이 적합한지, 그리고 구체적으로 조건을 어떻게 하는 게 더 확실하게 분류하고, 신호 생성을 할 수 있는지, 그리고 개발을 어떻게 진행하는 게 좋을지 알려줘.

말씀하신 전략은 **"상태(레짐) 전이 + 전이 직후의 모멘텀/반전 확인(보조지표)"** 구조라서, 퀀트에서 꽤 설득력 있는 뼈대입니다. 다만 성공하려면 ① **상태(15개 그룹) 정의를 흔들리지 않게** 만들고, ② **전이(하락/횡보→상승)만으로 매수하지 말고 '확인(confirmation)' 조건**을 붙이고, ③ **전이 확률/체류시간(마코프 체인 성격)**을 "필터"로 써야 합니다. 아래처럼 잡으면 구현도 깔끔하고 성능도 안정적으로 나옵니다.

---

1) "15개 그룹" 분류를 더 확실하게 만드는 방법

A. "방향(상승/횡보/하락)"을 단순 수익률 하나로 하지 마세요

기간별로 단순 수익률만 보면 횡보/추세 구분이 흔들립니다. 아래 3요소 중 2개 이상을 써서 **추세 강도**까지 반영하는 게 좋습니다.

- **기간 수익률**: $R_T = \frac{P_t}{P_{t-T}} - 1$
- **추세 기울기(회귀 slope)**: 최근 T일 log(price)에 선형회귀 → slope
- **추세 신뢰도**: 회귀 R^2 또는 t-stat(기울기 유의성)
- (보조) **변동성 대비 수익**: $\frac{R_T}{\sigma_T}$ (샤프 유사)

B. 3방향 판정 규칙(추천)

기간 T마다 아래처럼 **명확한 임계값**을 둡니다. (숫자는 시작점이고, 백테스트로 튜닝)

- **상승(Up)**:
  - $R_T > +a_T$ **그리고** slope > 0 **그리고** $R^2 > r_T$
- **하락(Down)**:
  - $R_T < -a_T$ **그리고** slope < 0 **그리고** $R^2 > r_T$
- **횡보(Sideways)**:
  - 그 외 (혹은 $|R_T| \le a_T$ 또는 R^2가 낮으면 강제로 횡보)

임계값 a_T는 기간이 길수록 크게 잡는 게 자연스럽습니다. 예:

- 1년(252): 15~25%
- 6개월(126): 8~15%
- 3개월(63): 5~10%
- 3주(15): 2~5%
- 3일(3): 0.8~2%

또 하나 중요한 팁: **임계값을 고정 %가 아니라 "유니버스 내 분위수(예: 상/하위 35%)"로 두면** 장세 변화에 둔감해집니다.

---

2) 상태 전이(셀 이동) 추적: "그럴듯"에서 "쓸모"로 바꾸는 핵심

A. 상태를 15개 "셀"로 보되, 실제로는 "전이 신뢰도"가 핵심입니다

전이 자체는 노이즈가 많습니다. 그래서 아래 2개를 같이 저장/사용하세요.

1. **체류시간(sojourn time)**: 한 상태에 평균 몇 일 머무는지
2. **전이확률(transition matrix)**: A→B로 갈 확률

이걸로 "하락/횡보→상승" 중에서도 **의미 있는 전이**를 골라냅니다.

B. "하락/횡보→상승" 매수 후보를 고르는 추천 조건

아래 중 2~3개를 동시에 만족할 때만 "상승 진입"으로 인정하세요.

- **연속성 조건**: 상승 상태가 **N일 연속 유지** (예: 3주 셀에서 Up이 3일 연속)
- **다중 기간 합의**: 단기만 Up 말고 **3주 Up + 3개월 Side/Up** 같이 상위 기간과 충돌이 적을 것
- **전이 희귀도 필터**: 과거 전이확률이 너무 낮은(=우연) 전이는 제외
  - 예: $P(\text{Down} \to \text{Up})$가 종목별 과거에서 하위 20%면 제외

- **하락 깊이 필터**: 직전 Down의 낙폭이 과도하면(급락 반등 노이즈) 제외 또는 별도 버킷

즉 "전이 발생"을 트리거로 쓰되, **'상승에 안착했는가' 확인**을 하고 들어가야 합니다.

---

3) RSI, MACD로 신호 만들 때의 추천 구조(전이 + 확인 + 청산)

A. 매수(BUY): "전이 발생"은 전제조건, RSI/MACD는 "확인"

추천 2안 중 하나로 단순하게 가세요.

안1) 추세형(성공률 안정적)

- 전제: (하락/횡보→상승) 전이 + 상승 연속 N일
- 매수 확인:
  - MACD line이 Signal 상향 돌파(또는 Histogram 0 상향)
  - RSI(14) > 50 상향 돌파

안2) 반전형(초기 진입 빠르지만 노이즈 큼)

- 전제: (하락→횡보→상승) "2단 전이" 또는 3주 Up 첫 진입
- 매수 확인:
  - RSI가 30~45 구간에서 상승 전환(예: RSI 5일 최저 후 상향)
  - MACD Histogram이 "덜 음수"로 2~3일 연속 개선

B. 매도(SELL): "상승 훼손" 또는 "과열"

- 기본(추세 훼손):
  - 3주 셀 Up → Side/Down 전이 **그리고** RSI < 50 하향
- 과열 청산:
  - RSI > 70에서 꺾임(고점 대비 RSI 하락)
  - 또는 MACD Histogram 감소 3일 연속(모멘텀 약화)

C. 보유(HOLD): 규칙을 간단히

- 상승 셀 유지 + RSI 50~70 + MACD 양(또는 개선)이면 HOLD
- 횡보 셀로 내려가면 "추가 확인 없으면 신규매수 금지" 같은 룰을 두면 거래가 줄고 성과가 안정됩니다.

---

4) 개발을 어떻게 진행하면 좋은가 (현실적인 구현 순서)

1단계: "상태 분류기(state classifier)"부터 독립 모듈로

- 입력: 가격 시계열(일봉), 기간 T 목록(252,126,63,15,3)
- 출력: 날짜별 상태(Up/Side/Down) × 기간별 = 5개 상태 라벨

2단계: "상태 전이 추적기(transition tracker)"

- 종목별로 날짜별 상태 시퀀스에서
  - 체류시간 분포
  - 전이 카운트 / 전이확률 행렬
  - "Down/Side → Up 후보 전이" 이벤트 리스트 생성

3단계: "시그널 엔진(signal engine)"

- 전이 이벤트가 발생한 날을 기준으로
  - RSI/MACD 계산
  - BUY/SELL/HOLD 판정
- 여기서 **룰을 최대한 단순하게**(나중에 튜닝)

4단계: 백테스트 통합

- 기존 전략은 그대로 두고,
- 새 전략은 **별도 전략 클래스로 붙여서** 동일한 백테스트 러너에서 비교 가능하게.
- 지표: Sharpe / MDD / CAGR는 기존 기준 그대로 두고, 추가로
  - 승률, 평균 보유기간, 거래당 기대수익, 슬리피지 민감도

---

5) 이 전략의 "가장 큰 리스크"와 방지책

- **리스크1: 분류가 흔들려서 상태 전이가 너무 잦아짐 → 과매매**
  - 방지: 상승 "연속 N일" + "다중 기간 합의" 필터
- **리스크2: 전이 확률이 종목마다 너무 불안정(표본 부족)**
  - 방지: 종목별 추정 + 유니버스 공통 추정(베이지안/가중 평균) 혼합
- **리스크3: 3일/3주 같은 초단기 셀이 노이즈**
  - 방지: 초단기는 '진입 타이밍'이 아니라 '진입 금지 필터'로 쓰는 게 오히려 안정적입니다

원하시면, 지금 구조(백테스트 러너/전략 클래스/데이터 로더)에 맞춰서 `state_classifier.py`, `transition_tracker.py`, `strategy_regime_transition.py` 같은 식으로 "새 전략만" 깔끔하게 추가하는 설계를 바로 제시해 드리겠습니다. (기존 전략은 손대지 않는 방향으로요.)

출처

네 말이 매우 합리적이야. 먼저, 내가 얘기한 그룹(class)의 분류에 대한 개념을 너와 내가 좀 더 확실하게 동의할 수 있게 만드는 게 제일 중요할 것 같아. 1년, 6개월, 3개월, 3주, 3일의 기간동안 상승, 보합, 하락 으로 분류하는 걸 어떤 지표들을 기준으로 할 건지 부터 정해야 겠어. 퀀트투자에서는 이런 기간동안의 주가 흐름을 어떤 기준으로 분류하는지 알려줘.

26s 동안 생각함 ›

퀀트(특히 "레짐/추세(Trend) vs 횡보(Range)"를 다루는 전략)에서 **기간별 주가 흐름을 상승·횡보·하락으로 분류**할 때는, 대체로 아래 4계열 중 하나(또는 2~3개 조합)를 씁니다. 핵심은 "수익률(방향)"만 보지 말고 **추세의 '강도/신뢰도'를 같이** 보자는 것입니다.

---

1) 가장 표준적인 분류 프레임 4가지

A. 이동평균(MA) 필터 기반: "가격이 장기 평균 위/아래인가"

가장 널리 쓰는 시장 국면(상승/하락) 필터입니다. 예: 10개월(또는 200일) SMA 위면 위험자산 보유, 아래면 축소 같은 규칙이 대표적입니다. SSRN +1

- **Up(상승)**: 가격 > MA(T) (또는 단기MA > 장기MA)
- **Down(하락)**: 가격 < MA(T)
- **Side(횡보)**: 가격이 MA 주변에서 빈번히 교차(혹은 밴드 폭 내)

장점: 구현 쉬움, 안정적. 단점: 횡보 구간에서 잦은 교차(휘둘림).

B. 회귀 기울기(slope) + 유의성(t-stat / R^2): "추세가 통계적으로 '있나'"

최근 T일의 log(price)를 시간에 대해 선형회귀해 **기울기 부호(방향)**와 **t-stat 또는 R^2(신뢰도)**로 분류합니다. 트렌드/타임시리즈 모멘텀 문헌에서도 비슷한 형태로 추세를 계량화합니다. 사이언스다이렉트 +2

- **Up**: slope > 0 AND |t| ≥ 임계값(예: 2.0) 또는 R^2 ≥ 임계값
- **Down**: slope < 0 AND |t| ≥ 임계값 또는 R^2 ≥ 임계값
- **Side**: |t|가 작거나 R^2가 낮음(=방향성이 통계적으로 약함)

장점: "횡보"를 깔끔히 정의 가능. 단점: 짧은 기간(3일 등)은 표본이 너무 작아 회귀 기반은 부적합.

C. 수익률/변동성(Vol) 정규화: "상승인데 '의미 있게' 올랐나"

단순 수익률 대신 **변동성 대비 수익(샤프 유사)** 또는 Z-score를 사용합니다.

- $R_T = P_t / P_{t-T} - 1$
- $Z_T = R_T / \sigma_T$ (또는 롤링 표준편차로 정규화)
- **Up**: Z_T ≥ +z0
- **Down**: Z_T ≤ −z0
- **Side**: |Z_T| < z0

장점: 장세(변동성) 변화에 덜 민감. 단점: 횡보를 "낮은 방향성"으로만 보는 한계(추세는 있는데 변동성이 커서 Z가 낮게 나오는 경우).

D. ADX(추세 강도) + DI(방향): "추세 강도가 충분한가"

ADX는 "방향"이 아니라 **추세 강도**를 측정하고, +DI/-DI가 방향을 보완합니다. 실무에서 "추세장/횡보장 필터"로 자주 씁니다. chartschool.stoc... +2

- **Side 후보**: ADX < 20 (추세 약함)
- **Up/Down 방향**: +DI > −DI면 Up 쪽, 반대면 Down 쪽(ADX가 충분히 높을 때)

장점: 횡보/추세 구분에 특화. 단점: 지표 특성상 지연(lag)이 있음.

2) 질문하신 "5개 기간(1년/6개월/3개월/3주/3일)"에 현실적으로 권하는 조합

기간이 짧을수록 노이즈가 커지므로 "같은 지표를 그대로" 쓰기보다 **기간별로 역할을 분리**하는 게 성과가 안정적입니다.

- **1년 / 6개월 / 3개월 (장·중기 추세)**
  → *MA 필터* 또는 *회귀 slope+t-stat(R^2)* + (보조로) *Z-score*
  이유: 표본이 충분해서 추세 신뢰도 판단이 가능.
- **3주(15거래일) (단기 추세/전환 포착)**
  → *Z-score* + (가능하면) *단기 MA 기울기* 정도가 적절
  회귀도 가능하긴 하지만 임계값을 약하게 두고 "확인용"으로 쓰는 편이 안전합니다.

- **3일 (초단기 노이즈 구간)**
  → 회귀/ADX는 표본 부족으로 비추천.
  3일 수익률과 갭/급등락 필터(예: |R_3|가 너무 크면 이벤트로 분리) 같은 "진입 금지/주의" 용도가 더 실용적입니다.

---

3) "상승/횡보/하락"을 한 번에 합의하기 좋은 권장 정의(초안)

당장 귀하의 15셀 분류를 위해, 가장 합리적으로 합의하기 쉬운 형태는 아래입니다.

1. **방향(Up/Down)**: 기간 T의 회귀 slope 부호(또는 가격 vs MA)
2. **추세 신뢰도(Trend vs Side)**: R^2(또는 |t|) **또는** ADX

예시(장·중기 T=252,126,63):

- Up: slope>0 AND $R^2 \geq r0$
- Down: slope<0 AND $R^2 \geq r0$
- Side: $R^2 < r0$ (방향이 있어도 신뢰도 낮으면 횡보로)

단기(T=15):

- Up/Down: Z_T의 부호와 크기(|Z|≥z0)
- Side: |Z|<z0

초단기(T=3):

- 분류용이라기보다 "필터": 급등락/갭이면 신호 생성에서 제외(또는 별도 라벨)

---

4) 참고: 더 고급 접근(필요 시)

HMM 같은 모델로 "상승/하락/중립" 레짐을 잠재상태로 두는 방식도 연구·실무에 있습니다. 다만 지금 단계에선 15셀 구조와 결합하면 복잡도가 급증합니다. SSRN +1

원하시는 "서로 확실히 동의 가능한 분류 정의"를 빠르게 만들려면, 다음 2가지만 먼저 결정하시면 됩니다.

1. 장·중기(1년/6개월/3개월)는 **MA 기반**으로 갈지, **회귀 slope + R^2(t-stat)**로 갈지
2. "횡보"를 무엇으로 볼지: **R^2(또는 |t|) 기반** vs **ADX 기반**

결정이 내려지면, 제가 그 정의를 기준으로 **각 기간별 임계값(예: r0, z0, ADX cut)**을 "유니버스 분위수 기반"으로 자동 튜닝되게 설계해서, 장세 변화에도 흔들리지 않게 만들겠습니다.

출처

퀀트투자에 주가흐름을 분류할 때 내가 생각한 상승, 하락, 횡보 외에 다른 구분도 쓰는 게 있나?

7s 동안 생각함 ›

있습니다. 실무 퀀트에서는 "상승·하락·횡보(방향 3분류)"만으로는 **리스크(변동성), 추세의 질(트렌딩 vs 채널), 드로우다운 국면** 같은 중요한 차이가 섞여 버려서, 보통 아래처럼 **추가 축을 하나 더 얹어 4~6개 레짐**으로 씁니다.

---

1) 가장 흔한 확장: "방향 × 변동성" (4분면)

방향(상승/하락/중립)에 **변동성(고/저)**를 추가합니다.

- 저변동 상승: "Quiet Bull"
- 고변동 상승: "Turbulent Bull"
- 저변동 하락: "Quiet Bear"
- 고변동 하락: "Turbulent Bear"

이런 형태는 HMM(은닉 마코프)나 Markov Regime Switching, 혹은 단순 규칙(수익률 부호 + 변동성 임계)로도 자주 라벨링합니다. QuestDB +3

왜 쓰나: "상승"이라도 고변동이면 손절·포지션 사이징이 달라지고, "하락"이라도 저변동이면 방어/대기 전략이 달라지기 때문입니다.

---

2) "추세장 vs 횡보장"을 별도로 분리 (Trend / Range)

단순히 횡보(중립)로 묶지 않고, **"추세가 있는가(Trend) 없는가(Range)"**를 별도 레짐으로 둡니다. 대표적으로 ADX가 낮으면 추세 부재(횡보)로 보고, 추세가 있을 때는 방향을 DI(+DI/-DI)로 나눕니다. TradingView +1

- Trend Up / Trend Down
- Range(Consolidation) ← 여기서 평균회귀 전략이 잘 맞는 경우가 많음

---

3) “위험회피 vs 위험선호” (Risk-on / Risk-off)

특히 멀티에셋/ETF 운용에서 흔한 축입니다. 주식이 오르는지 자체보다 **위험자산 선호가 강한지**가 중요해서,

- Risk-on: 주식/하이일드/소형주 강세, 변동성(예: VIX) 낮음
- Risk-off: 안전자산 선호, 변동성 상승
  같은 식으로 정의합니다. (레짐 정의 예시로 언급) ResearchGate

---

4) “추세 지속 vs 추세 소진/전환 구간” (Continuation / Exhaustion)

트렌드 팔로잉 쪽에서는 추세를

- 형성(formation) → 지속(continuation) → 소진(exhaustion) → 전환(turning)
  처럼 세분하는 관점이 있고, “소진/전환”은 **상승이지만 모멘텀이 약해지는 상태**로 따로 다루기도 합니다. CME Group

---

5) 실무적으로는 “3분류 + 추가 라벨”이 가장 많이 쓰입니다

정리하면,

- 기본 방향 3분류(Up/Side/Down)는 유지하되
- **(1) 변동성 고/저** 또는 **(2) 추세 강도(Trend/Range)** 중 하나를 추가해
  4~6개 레짐으로 확장하는 게 가장 흔한 패턴입니다. QuestDB +1

귀하의 “15셀(기간×방향)”에 어떻게 적용하는 게 합리적인가

지금 설계(5개 기간 × 3방향)는 이미 해상도가 높습니다. 여기에 레짐을 무작정 늘리면 복잡도가 커지므로, 저는 다음이 현실적이라고 봅니다.

- **분류 라벨은 15셀 그대로 유지**
- 대신 각 셀에 “보조 속성”으로만 추가:
  - `vol_regime` (High/Low) 또는
  - `trend_strength` (ADX 등으로 Strong/Weak)

이렇게 하면 “상승 진입”을 잡을 때

- **상승이면서 저변동(안정 추세)**만 우선 매수
- **상승이지만 고변동(휩쏘 위험)**은 신호 강화 조건 필요
  같은 운영이 가능해집니다.

원하시면, 다음 단계로 “15셀 분류의 정식 정의(지표 세트 + 임계값을 분위수로 자동화)”를 제가 제안드리겠습니다. 가장 먼저는 **추세 정의를 MA 기반으로 할지, 회귀 slope+R²/t-stat 기반으로 할지**를 고르시면 이후 설계가 빠르게 정리됩니다.

출처

내가 얘기한 5가지 기간 구분은 퀀트에서 많이 사용하는 개념인가? 아니면 내가 지나치게 세밀하거나 드문드문 구분해서 일반적인 퀀트투자에서 구분하는 기간과 다른가?

13s 동안 생각함 ›

결론부터 말씀드리면, 선생님이 잡으신 **1년/6개월/3개월**은 퀀트에서 매우 “정석적인” 구간입니다. 다만 **3주/3일**은 (전통적인 주식 팩터·레짐 문헌 기준으로) **상대적으로 비표준적**이며, 보통은 **“1주(5거래일)~1개월(20~21거래일)”** 같은 단위를 더 자주 씁니다. 그래도 3주/3일을 “레짐 전환의 확인(confirmation)·노이즈 필터”로 쓰는 목적이라면 충분히 합리적입니다.

---

1) 퀀트에서 흔히 쓰는 “기간(lookback) 구간”은 어떻게 생겼나

A. 중기 모멘텀(가장 표준)

주식 모멘텀 연구·실무에서 가장 널리 쓰이는 축은 **과거 3~12개월 수익률**을 기반으로 한 “중기 모멘텀”입니다. bauer.uh.edu +1

- 실무 팩터 정의도 대개 6~12개월(종종 최근 1개월 제외) 형태가 많습니다.

B. 추세추종(타임시리즈 모멘텀)

자산군 전반(선물/지수 등)에서 **1~12개월** 범위의 추세(과거 수익이 미래 수익의 양(+)의 예측변수)가 반복적으로 보고됩니다. 사이언스다이렉트 +1

C. 단기 되돌림(Short-term reversal)

주식에서는 아주 짧은 구간에서 **1주~1개월** 단위의 되돌림(리버설) 현상이 많이 다뤄집니다. 즉 “단기”를 잡더라도 3일보다 **1주/1개월**이 문헌·실무에서 더 보편적입니다. Aalto University'... +1

2) 선생님 5구간(1년/6개월/3개월/3주/3일)은 "일반적"인가?

1년 / 6개월 / 3개월

- **매우 일반적**입니다.
- 중기 모멘텀(3~12개월)과 추세추종(1~12개월) 범위 안에 정확히 들어옵니다. bauer.uh.edu +1

3주(15거래일)

- **가능은 하지만 비표준 쪽**입니다.
- "단기"를 보통 1주(5일), 1개월(20~21일)로 많이 잡는 관행에 비해 15일은 애매한 중간 값입니다.
- 다만 선생님처럼 "전환 감지(transition confirmation)"용이면 15일도 충분히 실용적입니다.

3일

- **일반적인 레짐/팩터 구간으로는 드뭅니다.**
- 3일은 미시구조/이벤트/갭 영향이 커서 "추세 분류"로 쓰면 라벨이 흔들릴 가능성이 큽니다.
- 대신 **"진입 금지/주의 필터(급등락, 갭, 변동성 쇼크)"**로 쓰면 가치가 있습니다.

---

3) 권장 정리: "세밀함"을 성과로 연결하는 방식

선생님 구간을 유지하되, 역할을 이렇게 분리하면 일반 퀀트 관행과도 맞고 성능도 안정화됩니다.

- **1년/6개월/3개월:** 핵심 레짐(Up/Side/Down) 결정(메인 분류)
- **3주:** "전환 확인" (Up 진입이 진짜인지 확인)
- **3일:** "노이즈/이벤트 필터" (진입·추가매수·손절을 막거나 조정)

그리고 "3주/3일이 과연 최적인가"는 철학 문제가 아니라 **백테스트로 비교**하면 끝납니다. 현실적으로는 비교 후보를 이렇게 두는 것이 표준적입니다.

- 단기 후보: **5일(1주)**, **10일(2주)**, **21일(1개월)**
- 초단기 후보: **3일 vs 5일**(필터 성능 비교)

원하시면 다음 단계로, "기간별로 어떤 지표(수익률/회귀 slope+R^2/MA/변동성 정규화 등)를 쓸지"를 **딱 하나의 표준 규격**으로 확정해 드리겠습니다. 그 다음에 15셀(5×3)을 코드로 흔들림 없이 찍을 수 있습니다.

출처

좋아, 일단 기간구분은 내 생각을 반영해 주고, 추세 확인 정확성을 가장 높일 수 있는 방향으로 지표들을 구성해서 표준 규격을 만들어 줘.

아래는 선생님이 고정하신 **5개 기간(1년/6개월/3개월/3주/3일)**을 그대로 유지하면서, 퀀트 실무 관점에서 **추세(방향)와 추세의 신뢰도(Trend vs Noise)를 함께** 잡아 "상승/횡보/하락" 라벨을 최대한 흔들리지 않게 만드는 **표준 규격(standard spec)**입니다.
핵심은 **(1) 방향은 '기울기(추세)'로, (2) 횡보 판정은 '추세의 신뢰도'로, (3) 단기(3주/3일)는 '확인/필터' 역할로** 설계하는 것입니다.

---

0) 공통 전제(모든 기간 동일)

입력 데이터

- 일봉 종가(조정종가 권장), 최소 2~3년 이상 히스토리
- 거래정지/상장폐지/이상치 처리(가격 급변, 결측)

공통 계산(기간 T마다)

1. **로그가격 회귀 추세**
- $y_i = \ln(P_i)$, $x_i = i$ (최근 T일)
- OLS 회귀로 **slope**(기울기), **t-stat**(기울기 유의성), **R^2**(설명력) 산출
2. **기간 수익률**
- $R_T = \frac{P_t}{P_{t-T}} - 1$
3. **변동성(정규화용)**
- $\sigma_T = \mathrm{std}(\Delta \ln P)$ (최근 T일)

공통 원칙(정확성 우선)

- "방향"은 **slope 부호**로 결정 (단순 수익률보다 안정적)
- "횡보"는 **추세 신뢰도(t-stat 또는 R^2)**가 낮으면 강제
- 임계값은 고정값보다 **유니버스 분위수(quantile)**로 자동화(장세/변동성 변화 대응)

---

1) 기간별 표준 규격(5개 기간 고정)

아래 표에서 T는 거래일 기준입니다.

A. 1년(252) / 6개월(126) / 3개월(63): "메인 레짐"

이 3개는 선생님 전략의 "핵심 분류"로 씁니다.

지표 세트

- Direction: slope 부호
- Trend confidence: |t-stat| 또는 R²
- Noise 방지: 수익률 크기(|R_T|) 보조

판정 규칙(권장)

- **횡보(Side)**: $|t_T| < t_T^{*}$ (또는 $R_T^2 < r_T^{*}$)
- **상승(Up)**: $|t_T| \geq t_T^{*}$ AND slope$_T > 0$
- **하락(Down)**: $|t_T| \geq t_T^{*}$ AND slope$_T < 0$

임계값 설정(정확성 우선)

- t_T^{*}: 유니버스에서 |t|의 **상위 60~70% 컷**을 "추세 있음"으로 정의
  - 예: 매 리밸런스일에 종목 200개 기준으로 |t| 분위수 0.35~0.40을 컷으로 두고, 그 미만은 횡보
- (보강) $|R_T|$가 극단적으로 작은데 |t|만 큰 경우(완만한 우상향)는 그대로 Up으로 두되, 신호 엔진에서 **포지션 사이징**으로 반영

  이렇게 하면 "횡보"가 단순히 수익률이 0 근처인 상태가 아니라, **통계적으로 추세가 약한 상태**로 잡혀서 흔들림이 줄어듭니다.

B. 3주(15): "전환 확인(Confirmation) 레이어"

15일은 표본이 작아서 회귀 t-stat은 불안정합니다. 그래서 3주는 **짧은 확인용 규칙**이 더 정확합니다.

지표 세트

- 단기 모멘텀: $Z_{15} = R_{15}/\sigma_{15}$ (변동성 정규화 수익)
- 단기 MA 기울기(선택): SMA(5) - SMA(15) 또는 SMA(5) slope
- 단기 추세 안정성: 최근 15일 중 양(+)수익률 일수 비율 p_{pos}

판정 규칙(권장)

- **횡보(Side_15)**: $|Z_{15}| < z^{*}$ (또는 p_{pos}가 0.4~0.6 구간)
- **상승(Up_15)**: $Z_{15} \geq z^{*}$ AND $p_{pos} \geq p^{*}$
- **하락(Down_15)**: $Z_{15} \leq -z^{*}$ AND $p_{pos} \leq 1 - p^{*}$

임계값

- z^{*}: 유니버스 기준 |Z| 분위수 0.60~0.70
- p^{*}: 0.60(=15일 중 9일 이상 양봉)부터 시작

  요점: 3주 셀은 "전이 감지"가 아니라 **전이 후 '안착'했는지** 확인하는 용도로 쓰는 게 정확성이 가장 높습니다.

C. 3일(3): "노이즈/이벤트 필터(분류가 아니라 게이트)"

3일을 Up/Side/Down으로 깔끔히 분류하는 것은 정확성 측면에서 손해가 큽니다. 그래서 표준 규격에서는 3일을 **라벨**이 아니라 **게이트 변수**로 둡니다.

지표 세트

- 초단기 수익률: R_3
- 초단기 변동성 쇼크: $|R_3|/\sigma_{63}$ (중기 변동성 대비 급변)
- 갭/급등락 탐지(가능 시): 전일 대비 갭, 상한가/하한가, 거래량 급증

게이트 규칙(권장)

- **Shock = 1(주의/진입보류)** if
  - $|R_3| > q_{R3}^{*}$ (유니버스 상위 10~15% 급변)
    또는 $|R_3|/\sigma_{63} > s^{*}$ (예: 2.0 이상)
- **Shock = 0(정상)** otherwise

  결과적으로 3일은 "상승/하락"을 나누기보다, **신호 생성 시 과도한 단기 이벤트를 걸러** 추세 확인 정확도를 올립니다.

---

2) 최종 산출물(표준 라벨 구조)

각 종목, 각 날짜 t에서 다음을 산출합니다.

1. 메인 레짐 3개

- `state_252` ∈ {UP, SIDE, DOWN}
- `state_126` ∈ {UP, SIDE, DOWN}
- `state_63` ∈ {UP, SIDE, DOWN}

2. 확인 레이어
- `state_15` ∈ {UP, SIDE, DOWN}
3. 필터
- `shock_3` ∈ {0,1}

그리고 선생님이 원하시는 **15셀 분류**는 (예: 252/126/63/15/3 각각 UP/SIDE/DOWN)로 만들되,

- **실제 "전이 추적"의 기준 셀은 252/126/63 중심**
- 15는 확인
- 3은 필터
  이렇게 역할을 분리하는 것이 정확도 측면에서 최선입니다.

---

3) "추세 확인 정확성"을 더 끌어올리는 운영 규칙(권장)

1. **다중 기간 합의(Consensus)**
- 예: 매수 후보는 최소
  - `state_63` 가 DOWN/SIDE → UP 전이 발생
  - 동시에 `state_126` 이 SIDE 이상(=DOWN이면 제외)
  - 그리고 `state_15` 가 UP으로 "N일 연속"(예: 2~3일) 확인
2. **횡보 우선 판정(보수적)**
- 애매하면 UP/DOWN이 아니라 SIDE로 보내는 것이 정확도를 올립니다.
- 즉, SIDE는 "진짜 횡보"뿐 아니라 "추세 신뢰도 부족"을 포함하는 안전 라벨입니다.
3. **임계값은 고정치가 아니라 분위수**
- 장세가 바뀌면 고정치(예: t=2.0)는 바로 깨집니다.
- 유니버스 200개에서 매 리밸런스마다 분위수로 컷을 재산정하는 것이 가장 실무적입니다.

원하시면, 다음 단계로는 이 표준 규격을 그대로 코드로 옮기기 위해 **(1) t-stat/R^2 중 무엇을 "추세 신뢰도" 기준으로 쓸지 하나로 통일**하고, **(2) 분위수 컷(예: 0.35, 0.65 등) 초깃값을 확정**하면 됩니다.
저는 정확성 우선이면 **장·중기: |t-stat|**, 단기: **Z-score + 양(+)일수비**, 3일: **shock 게이트** 조합을 추천드립니다.

> 좋아, 그러면 3주와 3일은 그룹으로 분류 게 아니라 이벤트성 테크로 표현을 하는 게 어떨가?
> 3개의 레짐, 3개의 기간으로 9개의 그룹을 만들고 2가지 테그를 각 종목에 표시하는 걸로 하면 너무 복잡해 지나?

결론부터 말씀드리면, **3주·3일을 "그룹(레짐)"이 아니라 "태그(이벤트/필터)"로 분리하는 설계는 매우 합리적**이고, 오히려 **복잡도를 낮추면서(설명력은 유지)** 백테스트/운영 측면에서 더 안정적입니다.
그리고 **3개 기간×3개 레짐 = 9개 그룹 + 2개 태그** 구성은 "너무 복잡"한 수준이 아닙니다. 다만 **9개 그룹을 '동시에 3개를 붙이는 방식'**으로 쓰면 복잡해지고, **"주 레짐 1개 + 보조 레짐 2개"**처럼 계층화하면 단순해집니다.

---

1) 왜 "3주/3일은 태그"가 더 낫나

- **3일**은 표본이 너무 짧아 레짐(추세) 분류로 쓰면 노이즈가 커집니다. 반면, "급변/이벤트"를 잡는 태그로는 강력합니다.
- **3주**는 "전환 확인(안착)"에 최적입니다. 레짐을 하나 더 늘리기보다, **전환의 신뢰도를 부여하는 태그**로 쓰면 신호 품질이 좋아집니다.

즉, 레짐은 **중기(3개월)~장기(1년)**가 담당하고, 3주/3일은 **레짐을 '확인/차단'하는 보조 장치**가 됩니다.

---

2) 권장 아키텍처: "9개 그룹 + 2 태그" (간단한 계층 구조)

A. 레짐(그룹) 정의: 3개 기간 × 3개 방향

- 기간: **1년(252), 6개월(126), 3개월(63)**
- 레짐: **UP / SIDE / DOWN**
- 결과: 각 종목은 매일 **(state_252, state_126, state_63)** 3개를 가집니다.
- "9개 그룹"은 사실상 "각 기간별로 3분류"라는 뜻이고, 종목을 한 칸에 박아 넣는다면 27조합이 될 수 있습니다. 그래서 **주 레짐을 하나 정해 단순화**합니다.

B. 단순화 규칙(중요)

- **주 레짐(primary)**: `state_63` (3개월)을 기본 레짐으로 사용
- **상위 필터(secondary)**: `state_126` , `state_252` 는 "충돌 여부"만 체크
  - 예: `state_63=UP` 인데 `state_252=DOWN` 이면 "상승이더라도 장기 하락장 역추세"로 표시

이렇게 하면 운영상 **사람이 이해 가능한 단순한 구조**가 됩니다.

3) 2개 태그(이벤트) 설계안

Tag 1) `confirm_15` **: "전환/추세 확인 태그(3주)"**

역할: `state_63` 가 UP으로 바뀌는 순간에 **진짜로 올라타도 되는지** 확인.

예시 정의(정확성 우선):

- 최근 15일 $Z_{15} = R_{15}/\sigma_{15}$가 유니버스 상위 분위수(예: 60~70%) 이상
- AND 15일 중 양(+)수익 일수 비율 $p_{pos} \geq 0.60$
- AND (선택) 5일 SMA > 15일 SMA
- 아니면 0

Tag 2) `shock_3` **: "급변/이벤트 차단 태그(3일)"**

역할: **진입 직전/직후** 시장 미시 이벤트로 인한 휩쏘를 차단.

예시 정의:

- `shock_3 = 1` if
  - |R_3|가 유니버스 상위 10~15% 급변
  - 또는 |R_3| / σ_63 > 2.0 (중기 변동성 대비 쇼크)
- 아니면 0

---

4) 이 구조로 "복잡해지는가?"에 대한 판단

복잡도는 "변수 개수"가 아니라 "의사결정 규칙이 사람에게 설명 가능한가"로 봐야 합니다.

- **나쁜 복잡도**: 5기간×3방향을 모두 동일 레벨로 취급하고, 15셀 전이를 전부 추적하며, 그 위에 RSI/MACD까지 얹는 것
- **좋은 복잡도(권장)**:
  - 레짐(중장기): 3개월 중심 + 6개월/1년은 충돌 체크
  - 태그(단기): confirm/shock 두 개로 "들어갈지/피할지"만 결정
  - 매수/매도는 RSI/MACD로 "확인 후 실행"

후자는 구현도 단순하고, 백테스트에서 해석도 쉽습니다. 따라서 "복잡하다"기보다 **설계가 정리되는 방향**입니다.

---

5) 운영 규칙(매수/매도)까지 한 줄로 정리한 표준(초안)

- **매수 후보**: `state_63` 가 DOWN/SIDE → UP 전이
- **진입 허용**: `confirm_15=1` AND `shock_3=0` AND (`state_126` != DOWN)
- **매수 트리거(보조지표)**: RSI(14) 50 상향 + MACD 히스토그램 개선(또는 0 상향)
- **청산**: `state_63` 가 SIDE/DOWN으로 전이 OR RSI<50 하향 OR MACD 모멘텀 약화

---

6) 다음 개발 단계(추천)

1. `regime_classifier.py` : 252/126/63 레짐 산출(회귀 slope + |t| 기반)
2. `event_tags.py` : confirm_15, shock_3 산출
3. `strategy_regime_event.py` : "레짐 전이 + 태그 게이트 + RSI/MACD" 신호
4. 기존 백테스트 러너에 전략만 추가해 비교

정리하면, 선생님 제안("3주/3일은 태그")은 타당하고, 복잡도를 낮추는 방향입니다.
다음으로 **주 레짐을** `state_63(3개월)` **로 확정**하고, `confirm_15` 와 `shock_3` 의 **분위수 컷(초깃값)**만 정해두면 바로 코드로 들어갈 수 있습니다.

나는 종목들을 각 그룹별로 나누려고 하는데, '주 레짐' 둔다는 말은 무슨 뜻이지?

"주 레짐(primary regime)"은 **각 종목을 한 시점에 '대표로' 어디 그룹에 넣을지(=분류 기준을 하나로 고정)** 정하는 개념입니다.
지금처럼 기간을 3개(1년/6개월/3개월)로 두면, 종목 하나가 동시에 3개의 레짐 라벨을 갖게 됩니다. 이때 **그 3개 중 하나를 '대표 분류 축'으로 정해** 종목을 실제 그룹(바구니)로 나누는 방식이 "주 레짐을 둔다"는 뜻입니다.

---

1) 왜 주 레짐이 필요한가

기간별 레짐을 모두 "동일한 그룹 기준"으로 쓰면 문제가 생깁니다.

- 종목 A: 1년 DOWN / 6개월 SIDE / 3개월 UP
- 종목 B: 1년 UP / 6개월 UP / 3개월 SIDE

이렇게 되면 "A는 어느 그룹인가?"가 애매해집니다.
즉, 종목을 그룹별로 **명확히 나눠 담으려면**, 분류 기준이 하나는 있어야 합니다.

그래서

- **주 레짐(Primary)**: 실제로 종목을 3개 그룹(UP/SIDE/DOWN) 중 어디에 넣을지 결정하는 기준
- **보조 레짐(Secondary)**: 주 레짐을 보완하는 설명/필터(예: 장기 하락장 속 단기 반등인지)

로 역할을 나눕니다.

---

2) "주 레짐을 3개월로" 추천한 이유

실무적으로 그룹이 유용하려면 "그룹의 성격이 너무 느리지도, 너무 빠르지도" 않아야 합니다.

- 1년: 너무 느립니다(전환이 늦게 잡혀 기회를 놓치기 쉬움)
- 6개월: 다소 느린 편이지만 안정적
- 3개월: 전환과 안정성의 균형이 좋아 "그룹 운용"에 적합

그래서 **3개월(63일)을 주 레짐**으로 두면,

- 종목을 매일(또는 리밸런스 주기마다) UP/SIDE/DOWN으로 깔끔히 나눌 수 있고,
- 6개월/1년은 "현재 3개월 UP이 장기 추세와 일치하는가?" 같은 **품질 관리 필터**로 쓰기 좋습니다.

---

3) 주 레짐을 두면 "그룹 분류"가 어떻게 되나 (예시)

매 리밸런스일에 유니버스 200개를 다음처럼 나눕니다.

- **Group U (상승군)**: `state_63 = UP`
- **Group S (횡보군)**: `state_63 = SIDE`
- **Group D (하락군)**: `state_63 = DOWN`

그리고 각 그룹 안에서 태그로 다시 표시합니다(선생님이 원하신 방식).

- `long_conflict = 1` if `state_252 = DOWN` (장기 하락장 역추세)
- `mid_confirm = 1` if `state_126 = UP/SIDE` (중기 충돌 없음)

즉, "그룹은 하나로 깔끔히 나누고", 나머지는 "딱 필요한 정보만 태그로 보강"합니다.

---

4) "주 레짐"을 안 쓰고도 그룹을 나누는 대안

원하시면 다음 대안도 있습니다.

대안 A) 27개 조합 그룹(1년×6개월×3개월)

- (UP/DOWN/SIDE)^3 = 27개 그룹
- 장점: 정보가 풍부
- 단점: 그룹 수가 너무 많아 표본이 쪼개지고 해석이 어려움(200개면 그룹당 평균 7~8개 수준)

대안 B) 계층형(트리) 그룹

1년을 먼저 3분류 → 그 안에서 6개월 3분류 → 그 안에서 3개월 3분류

- 장점: 구조가 논리적
- 단점: 결국 27개로 가며, 운영이 복잡해짐

그래서 보통은 **주 레짐 1개 + 보조 레짐/태그**가 가장 실용적입니다.

---

5) 선생님 목적("그룹 이동 추적")과의 연결

"하락/횡보에서 상승으로 들어가는 종목을 찾는다"가 목적이면,

- 그룹은 **명확히 3개**로 나뉘어야 전이(Down→Up)가 깔끔히 집계됩니다.
- 이때 주 레짐을 하나로 정해놓으면 전이 행렬(transition matrix), 체류기간(sojourn time) 계산이 단순해지고 신뢰도도 올라갑니다.

원하시는 형태로 정리하면 다음 두 선택지 중 하나를 고르시면 됩니다(질문 없이 제가 추천까지 드리면):

- **추천안(실용/정확성 균형)**: 그룹 분류는 `state_63` 하나로(UP/SIDE/DOWN), `state_126/state_252` 는 태그로만 표시
- **고해상도안(연구용)**: 27조합 그룹까지 가되, 분석 단계에서만 쓰고 실제 매매는 3개월 주 레짐으로 단순화

다음 단계로는, 선생님이 "종목을 그룹별로 나누는 주기"를 **일 단위**로 할지 **주/월 리밸런스**로 할지에 따라 그룹 운영이 달라집니다. 다만 질문을 더 늘리지 않기 위해, 기본값은 **월 1회(또는 주 1회) 리밸런스**로 잡는 것이 일반적으로 안정적입니다.

레짐을 강상승, 상승, 보합, 하락, 강하락 이렇게 5그룹으로 구분해서 지금 네가 얘기한 걸 다시 정리해 줘.

아래는 **레짐을 5그룹(강상승/상승/보합/하락/강하락)**으로 확장했을 때의 표준 규격과 운영 개념을, 앞서 논의한 "주 레짐 + 보조 레짐 + 이벤트 태그" 구조로 다시 정리한 것입니다. 핵심은 **그룹(버킷) 분류는 중기(3개월)를 대표로**, 6개월/1년은 **충돌·품질 태그**로 쓰고, 3주/3일은 **이벤트 태그**로 쓰는 것입니다.

---

1) 레짐(그룹) 5단계 정의: "방향 + 강도(확신)"로 분리

공통 계산(기간 T)

- 로그가격 회귀(최근 T일): slope, t-stat(또는 R^2)
- 기간 수익률: R_T
- 변동성: σ_T
- 정규화 모멘텀: $Z_T = R_T/\sigma_T$ (보조)

5그룹 라벨(권장 정의)

기본 철학:

- **보합**은 "방향이 없다"가 아니라 **추세 신뢰도가 낮다**
- **강/약(강상승·상승, 강하락·하락)**은 추세 강도(|t| 또는 |Z|)로 결정

(A) 3개월·6개월·1년(63/126/252): 회귀 t-stat 기반(정확성 우선)

- **보합(N)**: $|t_T| < t_0(T)$
- **상승(U)**: $t_0(T) \le t_T < t_1(T)$
- **강상승(SU)**: $t_T \ge t_1(T)$
- **하락(D)**: $-t_1(T) < t_T \le -t_0(T)$
- **강하락(SD)**: $t_T \le -t_1(T)$

여기서 $t_0(T), t_1(T)$는 고정값보다 **유니버스 분위수로 자동 산정**을 권합니다.

- 예: 매 리밸런스일에 유니버스 200개에서
  - t_0: |t|의 40% 분위수(그 미만은 보합)
  - t_1: |t|의 75% 분위수(그 이상은 강 추세)

이 방식이면 장세가 바뀌어도 "강/약/보합" 비중이 자동으로 안정됩니다.

---

2) "주 레짐(Primary)"과 종목 그룹 나누기: 5그룹으로 깔끔히

종목을 "각 그룹별로 나누려면" 대표 기준이 하나 필요합니다. 5그룹에서도 동일합니다.

추천: 주 레짐 = 3개월(63일) 5그룹

- **Group SU**: `regime_63 = 강상승`
- **Group U**: `regime_63 = 상승`
- **Group N**: `regime_63 = 보합`
- **Group D**: `regime_63 = 하락`
- **Group SD**: `regime_63 = 강하락`

이렇게 하면 유니버스 200개를 매 시점에 **반드시 5개 바구니 중 하나**에 넣을 수 있고, 전이 추적(하락→상승 등)도 단순해집니다.

---

3) 보조 레짐(Secondary): 6개월/1년은 "태그"로만 사용

3개월 그룹을 기준으로 종목을 나누되, 6개월/1년은 그룹을 쪼개지 말고 **품질·리스크 태그**로 둡니다.

예시 태그:

- `mid_trend_ok = 1` if `regime_126` ∈ {상승, 강상승, 보합} (중기 충돌 없음)
- `long_trend_ok = 1` if `regime_252` ∈ {상승, 강상승, 보합}
- `long_conflict = 1` if `regime_252` ∈ {하락, 강하락} (장기 역추세 위험)

이렇게 하면 "3개월 상승"이라도

- 장기 강하락이면 "역추세 반등 그룹"임을 즉시 표시할 수 있습니다.

---

4) 3주/3일: 그룹이 아니라 "이벤트 태그" (5그룹 구조에서도 동일)

Tag 1) `confirm_15` (3주 확인 태그)

역할: 3개월 레짐이 **하락/보합 → 상승 계열로 넘어갈 때** "안착 확인"

권장 정의(예):

- Z_{15}가 유니버스 상위 60~70% 이상
- 15일 중 양(+)수익률 일수 비율 $p_{pos} \geq 0.60$
- (선택) SMA5 > SMA15

→ 만족하면 `confirm_15=1`, 아니면 0

Tag 2) `shock_3` **(3일 쇼크 태그)**

역할: 단기 이벤트로 인한 휩쏘 차단

권장 정의(예):

- |R_3|가 유니버스 상위 10~15% 급변
- 또는 |R_3|/σ_63 > 2.0

→ 만족하면 `shock_3=1`, 아니면 0

---

5) 전이(셀 이동) 추적: 5그룹 버전으로 단순화

주 레짐(3개월 5그룹)만으로 전이를 추적하면 깔끔합니다.

- 상태공간: {SD, D, N, U, SU}
- 전이 이벤트(관심 대상):
  - **SD/D → U/SU** (하락 계열에서 상승 계열로 진입)
  - **N → U/SU** (보합 돌파)
- 전이 신뢰도 강화:
  - `confirm_15=1` 이어야 "유효 전이"로 인정
  - `shock_3=0` 이어야 "진입 가능"으로 인정

---

6) 신호 생성(RSI/MACD)은 "레짐+태그 게이트 이후"에만 실행

운영 규칙(간단 표준):

- **매수 후보**: `regime_63` 가 {SD, D, N} → {U, SU}로 전이
- **진입 허용 게이트**: `confirm_15=1` AND `shock_3=0` AND `long_conflict=0` (또는 완화)
- **매수 트리거**: RSI(14) 50 상향 + MACD 히스토그램 개선(또는 0 상향)
- **청산**: `regime_63` 가 {N, D, SD}로 하향 전이 OR RSI<50 하향 OR 모멘텀 약화

---

7) 구현 산출물(데이터 구조)

각 종목/일자에 아래만 저장하면 됩니다.

- `regime_63` (5그룹) ← 종목 그룹 분류의 기준
- `regime_126` (5그룹) ← 태그 생성용
- `regime_252` (5그룹) ← 태그 생성용
- `confirm_15` (0/1)
- `shock_3` (0/1)

이 정도면 복잡하지 않습니다. 오히려 "그룹은 5개로 단순", "보조 정보는 태그"라서 해석이 쉽습니다.

원하시면 다음 단계로, **t-stat 분위수 컷(예: 40%/75%)**을 초기값으로 고정하고, 유니버스 200개 기준으로 매 리밸런스일마다 자동 계산하도록 "정식 스펙 문서(표) + 의사코드"까지 바로 만들어 드리겠습니다.

내가 궁극적으로 원하는 건
1. 특정 종목의 주가 흐름이 어떤 레짐으로 변해왔는지, 어떤 그룹에, 어느정도 기간동안 속해 있었는지 그 히스토리를 관리해서 투자나 매도할 종목을 걸러내는 게 첫번째 목적이고
2. 특정 종목의 주식 매매 타이밍을 잡기 위해 그런 히스토리 관리가 유용한지, 어떤 지표들을 써야 하는 건지 알고 싶은 거야.
내가 지금 너랑 나눈 대화가 내가 원하는 목적에 부합되는 건지 알려줘.

네, 지금까지 논의한 방향은 선생님이 말씀하신 두 목적에 **대체로 부합**합니다. 다만 "부합한다"에서 끝내면 실무적으로 애매하니, 목적별로 **무엇이 충족되고 / 무엇을 추가로 명확히 해야 하는지**를 기준으로 냉정하게 정리해 드리겠습니다.

---

1) 목적 1에 대한 부합 여부

"레짐 히스토리(어떤 그룹에, 얼마나 오래 있었는지, 어떻게 이동했는지)로 종목을 걸러낸다."

지금 논의가 충족하는 것

- **레짐을 5그룹(강상승/상승/보합/하락/강하락)**으로 정의하고,
- **3개 기간(1년/6개월/3개월)**에서 레짐을 계산하며,
- 그중 **3개월을 '주 레짐'**으로 종목을 실제 그룹으로 나누고,
- 종목별로 **체류기간(sojourn time)**과 **전이(transition)**를 추적하자는 구조는,
  말씀하신 "히스토리 관리 기반 필터링"에 정확히 맞습니다.

즉, 선생님이 원하는 1번 목적(히스토리 기반 선별)은 이 설계로 바로 구현 가능합니다.

추가로 반드시 명확히 해야 하는 것(핵심)

히스토리를 "관리"해서 필터로 쓰려면, 레짐 데이터에서 최소 아래 KPI를 뽑아야 합니다.

- **최근 N일 중 레짐 분포**(예: 최근 126일 중 상승계열 비중)
- **최근 전이 패턴**(예: 하락→보합→상승 같은 2단 전이 빈도)
- **평균 체류기간**(상승 상태에 평균 몇 일 머무는지)
- **전이 확률/강도**(하락계열→상승계열로 넘어갈 "성향")

이 KPI가 있어야 "걸러낸다"가 계량화됩니다.
여기까지가 1번 목적을 실전으로 만들기 위한 마지막 관문입니다.

---

2) 목적 2에 대한 부합 여부

"매매 타이밍을 잡는 데 레짐 히스토리가 유용한가? 어떤 지표를 써야 하나?"

레짐 히스토리가 타이밍에 유용한가?

유용합니다. 단, 방식이 중요합니다.

- 레짐(특히 1년/6개월/3개월)은 본질적으로 **저주파(느린) 신호**라서
  '진입/청산의 날'을 정확히 찍는 도구가 아니라,
  "지금은 매수 신호가 잘 먹히는 환경인가 / 역추세 위험이 큰가" 같은 **타이밍의 '필터/컨텍스트'** 역할이 가장 강합니다.

그래서 우리가 "3주 confirm / 3일 shock" 같은 태그를 추가한 것은,
레짐을 타이밍에 연결하기 위한 **정석적인 보완**입니다.

지금 논의가 충족하는 것

- 레짐은 **환경 필터(투자 가능/불가, 역추세 위험)**로 쓰고,
- 3주(confirm)와 3일(shock)을 **진입 허용/차단**으로 쓰고,
- 그 다음 RSI/MACD로 **실제 매수/매도 트리거**를 만든다는 구조는,
  "레짐(저주파) + 트리거(고주파)" 결합이라서 타이밍 설계에 맞습니다.

즉, 2번 목적(타이밍)도 방향은 맞습니다.

다만, 반드시 짚고 넘어갈 현실적 한계

- RSI/MACD는 "정답 지표"가 아니라, 레짐 필터가 없으면 **횡보장에서 과다 신호**가 나고,
- 레짐만으로 타이밍을 잡으려 하면 **진입이 늦어지는 문제**가 생깁니다.

그래서 "레짐 히스토리 자체로 타이밍을 찍는다"가 아니라,

- **레짐 히스토리로 매매 가능한 후보/국면을 제한**하고,
- 후보에 대해서만 RSI/MACD(또는 다른 트리거)로 타이밍을 잡는 구조가 가장 합리적입니다.

현재 대화는 정확히 그 구조로 가고 있습니다.

---

3) 지금까지 대화가 "목적에 부합한다"의 결론

- **1번(히스토리 관리 → 종목 필터링): 부합합니다.**
  다만 다음 단계는 "필터 KPI를 무엇으로 확정할지"입니다.
- **2번(타이밍): 부합합니다.**
  다만 레짐은 타이밍 '결정'이 아니라 '조건부 게이트'로 쓰는 게 핵심입니다.

---

4) 다음 단계(목적에 맞게 바로 이어가려면)

질문을 늘리지 않고, 제가 권하는 실행 순서를 바로 드리면:

1. **레짐 히스토리 KPI 5개를 고정**
   - 예: (상승계열 비중, 최근 1회 전이 유형, 평균 체류기간, 하락→상승 전이 빈도, 장기 충돌 여부)
2. KPI 조합으로 **"투자 후보 점수(0~100)"**를 만든다
3. 점수 상위 종목에만 **RSI/MACD 트리거**를 적용해 진입/청산

이렇게 하면 "히스토리 관리"가 단순 기록이 아니라, 타이밍과 연결되는 **실제 의사결정 엔진**이 됩니다.

원하시면, 위 KPI 5개를 선생님 스타일(객관적/직설)로 **정량 정의(계산식, 기간, 임계값)**까지 확정해서 "필터링 스펙"으로 문서화해 드리겠습니다.

필터 KPI는 개발을 진행하면서 결정하면 될 것 같고, 퀀트투자에서 매매 타이밍을 잡는데 효과적인 전략이 뭔지 알려줘

12s 동안 생각함 ›

퀀트에서 "매매 타이밍"을 잘 잡는다고 평가받는 방법은, 결국 **가격이 가진 반복 패턴(추세 지속, 단기 되돌림, 변동성 확장)**을 이용하는 전략들입니다. 단일 지표가 만능인 경우는 드물고, 실전에서는 보통 **레짐(환경) 필터 + 타이밍 트리거** 조합이 가장 안정적입니다. (지금 선생님이 설계 중인 구조가 정확히 이 방향입니다.)

아래는 **타이밍 성능이 검증되어 널리 쓰이는 대표 전략**들입니다.

---

1) 추세추종(Trend Following) / 타임시리즈 모멘텀

핵심: 오르는 자산은 더 오르고, 내리는 자산은 더 내리는 "추세 지속"을 이용해 **진입·청산 타이밍**을 잡습니다.

- 연구적으로도 1~12개월 구간에서 추세 지속(타임시리즈 모멘텀)이 광범위 자산에서 관측됩니다. ScienceDirect +1
- 단순 규칙(예: 과거 n개월 수익률이 양(+)이면 롱, 음(-)이면 숏)도 동작하며, 이동평균 기반 규칙과 연결됩니다. 그대안의작은호...

대표 트리거(타이밍)

- **MA 필터**: 가격이 장기 MA 위면 보유/진입, 아래면 청산/회피
- **돌파형**: 최근 N일 최고가(또는 Donchian 상단) 돌파 시 진입, 하단 이탈 시 청산
- **ATR 기반 트레일링 스탑**: 추세 유지 중 손절/이익실현 타이밍을 자동화

장점: 큰 추세를 먹고, 위기 구간에서 하방 노출을 줄이기 쉬움
단점: 횡보장에서 잦은 손절(휩쏘)

참고로, "10개월 SMA(월간 200일 SMA 유사) 위/아래" 같은 단순 추세 필터가 다양한 자산에서 위험(변동성/드로우다운)을 낮춘다는 실증이 있습니다. mebfaber.com +1

---

2) 모멘텀(상대강도) 기반 리밸런싱: Cross-sectional Momentum

핵심: 같은 시점에 여러 종목을 놓고 "최근 수익률이 더 좋은 종목을 사고, 나쁜 종목을 판다"는 방식입니다.

- 주식 시장에서 "위너 매수/루저 매도" 모멘텀 효과는 고전적으로 잘 알려져 있습니다. bauer.uh.edu +1

타이밍 방식(실무형)

- 월 1회/주 1회 리밸런스(매일 사고파는 게 아니라 "정기 교체")
- 상위 k% 롱(또는 롱온리면 상위만), 하위는 제외/현금화

장점: 단순하고 확률적으로 강한 편
단점: 급격한 반전(리버설) 구간에 약함, 거래비용/회전율 관리 필요

---

3) 단기 되돌림(Short-term Reversal) / 평균회귀(Mean Reversion)

핵심: 단기 급등·급락은 일정 부분 되돌림이 발생한다는 성질을 이용해 **타이밍을 역방향으로** 잡습니다.

- 단기 반전(contrarian/return reversal) 관련 연구 흐름은 오래되었습니다. JSTOR +1

대표 트리거

- RSI 과매도(예: RSI<30)에서 반등 시 매수, RSI 과매수에서 꺾임 시 매도
- 볼린저 밴드 하단 이탈 후 재진입 시 매수(또는 상단 반대)
- 단기 급락(예: 3~5일 수익률 하위 분위수) + 거래량 쇼크 완화 시 역매수

장점: 횡보장에 강한 경우가 많음
단점: 강하락 추세에서는 "떨어지는 칼"이 되기 쉬움(레짐 필터가 필수)

---

4) 변동성 확장/수축 기반 타이밍(Volatility Regime)

핵심: 큰 추세는 종종 "변동성 확장"과 함께 시작되고, 위험구간은 변동성 상승으로 표시됩니다.

- 타이밍 자체를 변동성(또는 ATR)로 잡기보다는, **포지션 사이징/진입 제한/손절 폭 조절**에 매우 효과적입니다.

대표 적용

- ATR이 급증(쇼크)하면 신규진입 금지 또는 사이즈 축소
- 변동성이 낮게 압축된 뒤(스퀴즈) 돌파 발생 시 추세 전략 가중

선생님 시스템에 맞는 "가장 효과적인" 결론

선생님 목적이 "레짐 히스토리 기반으로 종목을 걸러내고, 그 안에서 타이밍을 잡는 것"이라면, 실전적으로 가장 성과가 안정적인 조합은 보통 다음입니다.

- **레짐이 상승/강상승(또는 보합→상승 전이 확인)**인 종목에만
  → **추세추종/돌파형 타이밍** 적용 (MA/Donchian/ATR) ScienceDirect +1
- **레짐이 보합**인 종목에만
  → **평균회귀 타이밍** 적용 (RSI/볼밴) JSTOR +1
- **레짐이 하락/강하락**이면
  → 롱온리 기준으로는 "대부분 회피" (혹은 아주 제한된 반등 전략만)

즉, "타이밍에 효과적인 전략"은 하나가 아니라 **레짐별로 다릅니다**. 그래서 레짐 히스토리 설계가 타이밍 성능을 올리는 데 실제로 유효합니다.

원하시면, 선생님이 이미 정리하신 **5레짐(강상승/상승/보합/하락/강하락)** 기준으로 "각 레짐에서 어떤 타이밍 트리거를 쓰는 게 합리적인지"를 **표 1장(레짐→진입/청산/금지 규칙)**으로 바로 규격화해 드리겠습니다.

출처

응. 거기까지 정리해 주고 본격적인 개발에 들어가자

아래는 **5레짐(강상승/상승/보합/하락/강하락)**을 전제로, "레짐 히스토리 기반 종목 선별 + 레짐별 매매 타이밍"을 **한 장 규격**으로 정리한 것입니다. 이 규격대로 구현하면 곧바로 개발로 들어갈 수 있습니다.

---

1) 레짐별 타이밍 전략 표준 규격

공통 전제

- **레짐 산출 기간(그룹용)**: 1년(252), 6개월(126), 3개월(63)
- **주 레짐(Primary)**: `regime_63` (종목을 실제 그룹에 넣는 기준)
- **보조 레짐(Secondary, 태그)**: `regime_126`, `regime_252` 는 "충돌/품질" 표시용
- **이벤트 태그**
  - `confirm_15` (3주 전환/추세 안착 확인)
  - `shock_3` (3일 급변/이벤트 차단)

  레짐은 "환경/후보 필터", 타이밍은 "트리거(진입/청산)"로 분리합니다.

A. 레짐→전략 매핑(핵심 표)

① 강상승(SU: Strong Up)

- **의미**: 강한 추세 지속 구간(추세추종이 가장 잘 먹히는 구간)
- **진입(Entry) 게이트**
  - `shock_3 = 0`
  - `regime_126 ∈ {상승, 강상승, 보합}` (중기 충돌 없을 것)
- **진입 트리거(둘 중 하나)**
  - (추세형) **Donchian 20일 상단 돌파** 또는 **종가가 MA20 위에서 MA20 기울기 양(+)**
  - (모멘텀형) **MACD 히스토그램 0 상향(또는 개선 3일 연속)**
- **청산(Exit)**
  - ATR 기반 트레일링 스탑(예: 2~3 ATR) **또는**
  - `regime_63` 가 상승 이하로 하향 전이(= 상/보/하로 내려감) + RSI<50 하향 확인
- **비고**
  - 이 구간은 "익절을 너무 빨리 하지 말고" 추세를 길게 가져가는 게 유리합니다.

② 상승(U: Up)

- **의미**: 추세는 있으나 강도는 중간(휩쏘 가능성 존재)
- **진입 게이트**
  - `confirm_15 = 1` (추세 안착 확인)
  - `shock_3 = 0`
  - `regime_252` 가 강하락이면(역추세) 포지션 사이즈 축소 또는 진입 금지(정책 선택)
- **진입 트리거**
  - **RSI(14) 50 상향 돌파** + **MACD 개선**(히스토그램 증가 또는 시그널 상향)
- **청산**
  - RSI(14) 50 하향 돌파
  - 또는 `regime_63` 가 보합/하락으로 전이
- **비고**
  - 상승 구간은 "확인(15일)"이 들어가면 신호 품질이 크게 좋아집니다.

③ 보합(N: Neutral/Side)

- **의미**: 추세 신뢰도 낮음(평균회귀/스윙이 상대적으로 유리)

- **진입 게이트**
  - `shock_3 = 0`
  - 장기 강하락(강하락)과 결합된 보합은 "하락 중 쉼"일 수 있으니 보수적으로(선택)
- **진입 트리거(평균회귀)**
  - **RSI(14) < 30 → 30 상향 복귀**(과매도 탈출)
  - 또는 **볼린저 밴드 하단 이탈 후 재진입**
- **청산**
  - RSI 50~60 근처 도달 시 분할청산(보합은 추세가 없으니 목표를 짧게)
  - 또는 중앙선(밴드 중단/MA20) 회귀 시 청산
- **비고**
  - 보합에서 추세추종을 하면 잦은 손절이 나기 쉽습니다(전략 분리의 이유).

④ 하락(D: Down)

- **의미**: 롱온리 기준 "회피가 기본", 예외적으로 반등 트레이드만 제한적으로
- **정책(권장)**
  - 기본: 신규 진입 금지
  - 예외(선택): "단기 반등"만 허용
- **예외 진입 조건(매우 제한)**
  - `confirm_15 = 1` (전환 징후)
  - `regime_63` 가 하락이더라도 `regime_126` 이 보합 이상으로 완화(하락 둔화)
  - RSI(14) < 30에서 상향 반전 + MACD 히스토그램 개선
- **청산**
  - 짧게(중앙선/RSI 50 근처) 또는 손절 엄격(ATR 1.5~2)
- **비고**
  - 하락에서 평균회귀는 "떨어지는 칼" 리스크가 크므로, 레짐/태그 게이트가 없으면 금지에 가깝게 운용하는 게 안전합니다.

⑤ 강하락(SD: Strong Down)

- **의미**: 시스템적으로 위험구간
- **정책**
  - 신규 진입 금지(롱온리)
  - 기존 보유분은 방어/청산 우선(리스크 축소)
- **비고**
  - 이 구간은 "타이밍 전략"보다 "노출 관리"가 핵심입니다.

---

2) "레짐 히스토리"가 매매 타이밍에 실제로 어떻게 연결되는가

- 레짐 히스토리의 역할은 **(1) 어떤 전략을 적용할지 선택**하고, **(2) 신호의 품질을 높이는 게이트(진입 허용/차단)**를 제공하는 것입니다.
- 즉, "레짐=타이밍 트리거"가 아니라
  레짐(저주파)로 환경을 정하고 → RSI/MACD/돌파(고주파)로 타이밍을 찍는 구조가 가장 효과적입니다.
  지금까지 논의는 정확히 이 구조입니다.

---

3) 이제 개발로 들어가기 위한 작업 분해(바로 착수용)

모듈 3개 + 전략 1개로 시작하겠습니다

1. `regime_classifier.py`
- 입력: 가격(일봉)
- 출력: `regime_63`, `regime_126`, `regime_252` (5레짐)
- 방식: slope + t-stat 기반 + 분위수 컷(초깃값: 보합컷 40%, 강추세컷 75% 추천)
2. `event_tags.py`
- 출력: `confirm_15`, `shock_3`
- confirm_15: Z15 + 양(+)일수비(p_pos) 기반
- shock_3: |R3| 분위수 또는 |R3|/σ63 기반
3. `regime_history_store.py` (또는 기존 데이터 저장 모듈에 통합)
- 종목×일자별 레짐/태그를 Parquet/DB로 저장
- "어느 그룹에 얼마나 있었는지"는 여기서 집계 가능
4. `strategy_regime_timing.py`
- 위 표준 규격대로 레짐별로 트리거를 적용해 BUY/SELL/HOLD 생성
- 기존 백테스트 러너에 "새 전략"으로 추가(기존 전략은 유지)

원하시는 대로, 여기까지가 "정리"입니다.
다음 메시지에서 바로 **개발 착수**로 넘어가겠습니다. 우선 저는 `regime_classifier.py` 의 **5레짐 산출 스펙(분위수 컷 적용 방식, 컬럼명, 출력 포맷)**을 코드 수준으로 확정하고, 이어서 `event_tags.py` 까지 한 번에 붙이는 순서로 진행하겠습니다.

개발 착수하기 전에, 그동안 개발을 했던 결과물(코드)들을 확인해서, 이 투자전략 시스템을 개발할 때 사용할 수 있는 것과 새로 개발해야 하는 것들을 구분하자.

32s 동안 생각함 ›

좋습니다. 다만 한 가지는 분명히 말씀드려야 합니다. **제가 지금 이 대화창에서 선생님 PC의 `D:\Quant` 코드(로컬 파일)를 직접 열어 확인할 수는 없습니다.** 그래서 "실제 파일 기준으로" 재사용/신규를 확정하려면, **현재 코드 트리/핵심 파일 내용**을 선생님이 이 채팅에 공유해 주셔야 합니다(트리 출력 또는 zip 업로드).

그럼에도 개발 착수 전 단계로서, **퀀트 시스템에서 일반적으로 이미 만들어져 있을 가능성이 높은 것(재사용 후보)**과, **이번 신규 전략에 필수인 신규 개발 영역**을 먼저 명확히 구분해 드리겠습니다. 그리고 마지막에 **코드 인벤토리 뽑는 최소 명령**까지 드리겠습니다.

---

1) 재사용 가능한 것들(기존 결과물에서 가져올 가능성이 높은 영역)

아래는 "이미 선생님이 DART ETL/백테스트/DB를 구축해 왔다"는 전제에서, 이번 전략에도 그대로 쓰일 확률이 높은 컴포넌트들입니다.

A. 데이터/ETL 계층

- **DART ETL 파이프라인**
  - 상장사 마스터(dim_corp_listed 등), 재무 팩트 테이블(fs_annual 등), 정제/검증 스크립트
- **유니버스 선정 로직(200개 추출)**
  - 재무지표 기반 필터링/랭킹(품질/가치/성장 등), 결측/이상치 처리
- **가격 데이터 로더**
  - 종목별 일봉/수정주가 수집·캐싱·정합성 체크(거래정지, 상장폐지, 액면분할 등)

→ 이번 전략도 "유니버스 200개 + 일봉 가격"이 전제이므로, 여기서 새로 만들 것보다 **기존 것을 그대로 쓰는 게 정답**입니다.

B. 백테스트 엔진/성과 평가

- **백테스트 러너(포트폴리오 구성, 리밸런스, 거래비용, 슬리피지, 체결 규칙)**
- **성과 지표 산출(Sharpe/MDD/CAGR 등) 및 결과 저장**
- **리포트/시각화 기본 프레임(수익곡선, MDD, 거래 로그)**
- **로깅/설정(config) 구조**

→ 새 전략은 "신호 생성"만 바뀌는 것이고, 백테스트 실행 프레임은 동일하게 유지하는 게 개발 효율과 일관성에 유리합니다.

C. 공통 유틸/인프라

- 캘린더(거래일), 리밸런스 스케줄러
- 데이터 캐시(parquet/DB), 경로 규약
- 에러 핸들링/재시도/성능 최적화(벡터화, 멀티종목 루프 등)

---

2) 이번 전략에서 "신규 개발"이 필요한 것들(핵심 차별 요소)

선생님이 합의하신 구조(5레짐 + 히스토리 + 전이 + 타이밍)는 기존 일반 백테스트에 없는 기능이 많습니다. 아래는 신규로 들어가야 하는 모듈들입니다.

A. 레짐 분류기(핵심)

- `regime_classifier`
  - 기간: 63/126/252 (3개월/6개월/1년)
  - 레짐: 강상승/상승/보합/하락/강하락(5그룹)
  - 산출: 종목×일자별 레짐 라벨 + (가능하면) 기초 통계(t-stat, slope 등)

  기존 전략이 RSI/MACD 중심이었다면 "레짐 라벨링" 자체는 거의 없을 가능성이 큽니다.

B. 이벤트 태그(3주/3일)

- `confirm_15` (전환/추세 안착 확인)
- `shock_3` (급변 이벤트 차단)

C. 레짐 히스토리 저장/조회(목적 1의 핵심)

- 종목별로:
  - 레짐별 체류기간
  - 전이(transition) 횟수/확률
  - 최근 N일 레짐 분포/전이 패턴
- 이 히스토리를 "필터/랭킹"으로 뽑는 조회 API

D. 전이 기반 후보 추출 로직(목적 1→2 연결부)

- 관심 전이 정의: (하락/보합 → 상승계열 진입 등)
- 전이 이벤트 발생 시점 리스트업
- 전이 신뢰도(예: confirm_15 필수, shock_3 배제 등)

E. 레짐별 타이밍 전략(목적 2)

- 레짐(SU/U/N/D/SD)별로 서로 다른 트리거 적용
  - 추세형(돌파/MA/ATR) vs 평균회귀(RSI/볼밴) 분리
- 동일 백테스트 엔진에서 기존 전략과 "전략 클래스"로 병렬 비교 가능하게 구현

F. 레짐 시각화/리포트(검증 생산성)

- 종목별 "레짐 타임라인(색상 밴드)" + 가격 차트
- 전이 지점 표시, 매수/매도 신호 오버레이

---

3) "재사용 vs 신규"를 확정하기 위한 최소 점검 방법(선생님이 한 번만 실행/공유)

아래 2가지만 주시면, 제가 **파일 단위로** 재사용/신규를 확정해서 "개발 작업 분해(WBS)"까지 깔끔하게 자르겠습니다.

(1) 폴더 트리/파일 목록

PowerShell에서 프로젝트 루트에서:

powershell 코드 복사

```powershell
cd D:\Quant
tree /F /A > _snapshot_tree.txt
```

또는 파이썬 파일 목록만:

powershell 코드 복사

```powershell
cd D:\Quant
dir -Recurse -Filter *.py | Select-Object FullName | Out-File _snapshot_pyfiles.txt
```

(2) "백테스트 진입점"이 되는 파일 1~2개

예: `run_backtest_*.py` 또는 현재 쓰는 러너 파일(이름이 무엇이든) 내용을 그대로 붙여주시면 됩니다.

---

4) 제가 제안하는 분류 기준(확정 규칙)

코드를 받으면 다음 규칙으로 분류하겠습니다.

- **재사용(그대로 사용)**: 데이터 수집/정제/저장, 백테스트 엔진, 성과지표, 로깅/리포트 프레임
- **부분 재사용(확장)**: 시각화(신호 표시 방식), 기존 보조지표 계산 모듈(RSI/MACD 등)
- **신규(이번 전략 전용)**: 레짐 분류/태그/히스토리/전이/레짐별 타이밍 전략 모듈

원하시면 지금 바로 진행하겠습니다.
선생님께서는 위의 `_snapshot_tree.txt` (또는 `_snapshot_pyfiles.txt`) 내용을 채팅에 붙여주시거나, 프로젝트를 zip으로 올려주시면 됩니다. 그러면 제가 **파일별로 "재사용/수정/신규" 리스트를 확정**하고, 그 다음 메시지에서 곧바로 개발 착수(WBS → 모듈 설계 → 코드 작성)로 들어가겠습니다.

_snapshot_pyfiles.txt
문서

_snapshot_tree.txt
문서

이걸 보고 판단해 줘.

45s 동안 생각함 ›

According to a document from **2026-01-26**, 올려주신 스냅샷 기준으로 현재 코드베이스는 **(1) 데이터 수집/정비 → (2) 유니버스 구성 → (3) 팩터/지표 계산 → (4) 백테스트 엔진/리포트**의 "파이프라인 골격"은 이미 갖추고 있고, 이번에 하시려는 **레짐(상태) 히스토리·전이(transition) 기반 전략**은 그 위에 **"레짐 분류/저장/전이분석 레이어"**를 새로 얹는 형태가 가장 효율적입니다.

---

1) 이번 전략에 그대로 재사용 가능한 것들 (이미 갖춰진 골격)

A. 유니버스·가격 데이터 파이프라인

- **Top200 유니버스 생성**: `build_universe_top200_monthly.py` (top200 월별 구성) _snapshot_pyfiles
- **가격 수집/저장/업데이트**: `price_backfill.py`, `price_store.py`, `price_update_daily.py` _snapshot_pyfiles
- (필요 시) **키움 가격 수집기**: `kiwoom_prices.py` _snapshot_pyfiles

→ 이번 전략은 "각 종목의 기간별 레짐 분류"가 핵심이므로, **가격 데이터가 안정적으로 쌓이고 갱신되는 구조**가 중요합니다. 이 부분은 재사용 가치가 큽니다.

B. 지표/팩터 계산 모듈

- **기술지표 계산**: `technical_indicators.py` _snapshot_pyfiles
- **가격 기반 팩터**: `price_factors.py` _snapshot_pyfiles
- **재무(펀더멘털) 팩터**: `fundamental_factors.py` 및 연간 팩터 구축/정규화 스크립트 _snapshot_pyfiles

→ 레짐이 "상승 전환 후보"로 들어왔을 때, **RSI/MACD 등으로 타이밍 신호 생성**(3단계)을 하시려면 이 레이어는 그대로 씁니다.

C. 백테스트/리포트 인프라

- **백테스트 엔진**: `backtest_engine.py` _snapshot_pyfiles
- **성과 리포트**: `report_performance.py`, `performance_report.py` _snapshot_pyfiles _snapshot_pyfiles
- **포트폴리오 리밸런싱**: `portfolio_rebalancer.py` _snapshot_pyfiles

→ 이번 전략도 결국 "필터링된 후보를 매수/보유/매도 룰로 운영"하는 백테스트가 필요하니, 엔진/리포트는 유지하는 게 맞습니다.

---

2) "전략 실행 스크립트" 중 정리(리팩토링) 권장 영역

현재 `run_backtest_quality_dynuniv_ver_...` 처럼 **버전 파일이 누적**되어 있고, `broken_backup` 도 존재합니다 _snapshot_pyfiles .
이 상태로 새 전략을 붙이면, 전략별 실험이 늘어날수록 관리 비용이 커집니다.

- 권장 정리 원칙:
  1. `src/backtest/` 아래의 `run_*.py` 는 **experiments(또는 scripts)** 폴더로 이동
  2. 재사용 코드는 `src/` 의 **모듈(라이브러리)**로 고정
  3. "전략별 엔트리포인트"는 `run_전략명.py` 1개만 남기고 내부는 공통 엔진 호출

(이 정리는 새 전략 개발 속도에 직접 영향을 줍니다.)

---

3) 이번에 "새로 개발해야 하는 것들" (핵심 레이어)

사용자 목표(레짐 히스토리 관리 + 전이 기반 후보 발굴)를 구현하려면, 아래 3가지가 **필수 신규 모듈**입니다.

A. 레짐 분류기 (5단계: 강상승/상승/보합/하락/강하락)

- 입력: (종목, 기준일 t, lookback=1년/6개월/3개월) 가격 시계열
- 출력: 레짐 라벨 + 근거 지표(모멘텀, 추세기울기, 변동성, 드로우다운 등)

※ 이미 `analyze_regime_switches.py` 가 있으므로, 그 파일은 "개념 검증/초안"으로 재활용 가능성이 높습니다 _snapshot_pyfiles (다만 지금 레짐 정의가 5단계인지, 어떤 지표를 쓰는지는 파일을 열어봐야 확정됩니다).

B. 레짐 히스토리 저장소 (DB/파일)

- "특정 종목이 어떤 레짐에 **얼마나 머물렀는지**"를 계산하려면
  - (ticker, date, horizon, regime, score, metadata…) 형태의 테이블이 필요합니다.
- 이미 DART ETL 쪽에서 DB를 다루는 흐름이 있으니(`db_init.py` 등) DB 패턴은 재사용 가능합니다 _snapshot_pyfiles .

C. 전이(transition) 분석기

- 목표: "하락/보합 → 상승(또는 강상승)으로 들어가는 패턴"을 찾기
- 산출물:
  - 레짐 전이 행렬(마코프 전이확률)
  - 평균 체류기간(sojourn time)
  - 특정 전이 이벤트의 이후 수익률 분포(전이 후 +5/+20/+60 영업일)

---

4) 3주/3일을 "태그(이벤트)"로 빼는 안의 적합성

결론적으로, **복잡도를 줄이면서도 정보 손실이 거의 없는 합리적 설계**입니다.

- 1년/6개월/3개월: "구조적 레짐(메인 상태)" → 5단계 레짐 분류
- 3주/3일: "단기 이벤트 태그"
  - 예: `tag_3w = (최근 15영업일 고점 돌파/변동성 급증/갭상승 등)`
  - 예: `tag_3d = (RSI 급반등, 단기 추세선 돌파, 거래대금 스파이크 등)`

즉, **(메인 레짐 3개 기간 × 5단계 = 15개 상태)**는 그대로 유지하되, 단기(3주/3일)는 상태공간을 폭발시키지 않고 **태그로만 추가 정보**를 주는 형태가 운영상 훨씬 깔끔합니다.

5) 개발 진행 순서 (현 코드 기준 "최단 경로")

1. **레짐 표준 규격(정의) 확정**
   - 1년/6개월/3개월 각각에 대해 5단계 레짐을 "같은 규격"으로 산출
2. **레짐 히스토리 테이블 생성 & 일괄 생성(backfill)**
   - Top200 전체에 대해 과거 N년치 레짐 라벨을 일괄 생성/저장
3. **전이 분석 리포트**
   - "보합/하락 → 상승/강상승" 전이 이벤트를 랭킹(빈도·성공률·기대수익)
4. **타이밍 신호(지표 기반) 결합**
   - 이미 있는 `technical_indicators.py` 를 활용해 RSI/MACD 신호를 붙임 _snapshot_pyfiles
5. **백테스트 엔진에 새 전략 플러그인 추가**
   - `backtest_engine.py` 는 유지, 전략 모듈만 신규 추가 _snapshot_pyfiles

원하시면 다음 단계로, **"레짐 5단계 표준 규격(지표 조합 + 임계값 + 라벨 규칙)"**을 확정하는 문서/스펙을 먼저 만들고(코딩 전), 그 스펙대로 `src/regimes/` (신규) 아래에 모듈을 설계하는 방식으로 바로 착수하겠습니다.

출처

> 지금 현재 d:/quant 폴더에 있는 기존 투자전략 코드는 별도로 사용해야 하니까 그대로 두고, 하부에 d:/quant/regime 폴더를 만들어서 지금 새로 만드는 파일들을 여기에 넣으면 관리가 쉬울까?

네, **관리 관점에서는 그 방식이 유리**합니다. 기존 `D:\Quant` 의 투자전략/백테스트 코드는 그대로 두고, 새로 만드는 "레짐(상태) 기반 전략" 관련 코드를 `D:\Quant\regime` **아래에 독립적으로 모아두면** 다음 장점이 큽니다.

왜 쉬워지나

1. **기존 코드 비침투(Non-invasive)**
   - 기존 전략을 건드리지 않고 새 전략을 개발/실험할 수 있어 회귀(기존 성능 깨짐) 리스크가 줄어듭니다.
2. **모듈 경계가 명확**
   - "레짐 분류/히스토리/전이/태그/전략"이 하나의 서브시스템으로 묶여, 파일이 늘어나도 구조가 유지됩니다.
3. **실험 파일(run_*.py) 난립 방지**
   - 기존 루트에 흩어지면 `run_...ver_...py` 가 더 증가하는데, `regime` 아래로 모으면 실험 스크립트도 격리됩니다.

권장 폴더 구조(실전 운영 기준)

`D:\Quant\regime\` 아래를 "패키지(라이브러리) + 실행 스크립트 + 산출물"로 나누는 게 가장 깔끔합니다.

- `D:\Quant\regime\src\regime\`
  - `regime_classifier.py` (252/126/63 → 5레짐 산출)
  - `event_tags.py` (confirm_15, shock_3)
  - `transition_analyzer.py` (전이행렬, 체류기간, 전이 이벤트 리스트)
  - `history_store.py` (레짐 히스토리 저장/조회: parquet/DB)
  - `strategy_regime_timing.py` (레짐별 타이밍 신호 생성)
- `D:\Quant\regime\scripts\`
  - `run_build_regime_history.py` (Top200 전체 백필)
  - `run_transition_report.py` (전이 분석 리포트)
  - `run_backtest_regime_strategy.py` (기존 backtest_engine 호출)
- `D:\Quant\regime\data\` (레짐 히스토리 저장: parquet 등)
- `D:\Quant\regime\reports\`
- `D:\Quant\regime\logs\`

단, 꼭 조심할 점 2가지

1. **import 경로 충돌**
   - 기존 코드가 `src\` 를 기준으로 모듈을 import하고 있다면, `regime\src\` 도 같은 방식으로 맞추는 게 안전합니다.
   - 가장 흔한 실수는 "동일한 파일명/모듈명"이 두 군데 생겨서 엉뚱한 모듈이 import되는 것입니다.
2. **데이터/산출물 경로를 분리**

- 레짐 전략의 산출물(레짐 히스토리, 전이 리포트)은 `D:\Quant\regime\...` 로 고정해 두셔야 기존 결과물과 섞이지 않습니다.

제 결론

- "기존 코드는 그대로 두고 `D:\Quant\regime` 에 새 전략을 넣는 방식"은 **맞는 방향**입니다.
- 위처럼 `src/regime` **와** `scripts` **를 분리**하면, 앞으로 전략이 늘어나도 관리가 무너지지 않습니다.

원하시면 다음 단계로, 선생님 폴더 구조(현재 `backtest_engine.py` 가 어디에서 어떻게 import되는지)를 기준으로 `regime` **쪽에서 기존 엔진을 '호출만' 하도록** 경로/패키징을 정확히 맞춘 실행 스크립트 골격부터 바로 잡아드리겠습니다.

> 일단 regime 폴더는 만들었어. 근데, 지난번 마지막 개발을 진행할 때 DART에서 데이터 수집은 업데이트가 됐는데, 주가 데이터 수집이 제대로 안됐어. 그래서, 내 생각에 장기적으로 안정적인 주가 데이터 수집을 위해 내 키움증권 계정으로 API 접속해서 주가 데이터를 가져오는 코드를 만들어 놓은 게 있으니까 그걸 활용해서 주가 데이터 수집을 하면 좋을 것 같아.

1m 37s 동안 생각함 ›

제공해주신 프로젝트 스냅샷 기준으로, **DART 수집과 유니버스 구성/백테스트 골격은 이미 갖춰져 있고**, "주가 데이터 수집 레이어"만 안정화하면 다음 단계(레짐 분류/전이 추적)로 바로 갈 수 있는 상태입니다. 특히 **키움 기반 주가 수집 코드(kiwoom_prices.py)** 와 **가격 저장/업데이트 파이프라인(price_store.py, price_update_daily.py, price_backfill.py)** 이 이미 분리돼 있어서, "데이터 소스만 키움으로 교체/보강"하는 접근이 가장 비용 대비 효과가 큽니다. _snapshot_pyfiles

---

1) d:/quant/regime 폴더 분리는 좋은 선택입니다

- 기존 전략 코드는 그대로 두고, **새 전략(레짐 분류/전이/신호)을 regime 하위에서 개발**하는 방식이 관리 측면에서 합리적입니다.
- 다만 "데이터 수집(주가)"은 전략과 분리된 **공용 인프라 레이어**이므로, 가능하면 기존 `src/collectors/price/*` 를 유지하면서 **그 안에서 키움 소스로 안정화**하는 게 좋습니다. (regime 폴더로 수집 코드를 새로 복제하면, 나중에 두 파이프라인이 갈라져 유지보수 비용이 커집니다.)

---

2) 키움 주가 수집을 "장기적으로 안정적"으로 쓰는 핵심 포인트

키움 OpenAPI는 안정적이지만, 운영상 함정이 명확합니다. **성공하려면 구조를 이렇게 잡아야 합니다.**

A. "32비트 실행 문제"를 구조적으로 분리

키움 OpenAPI는 일반적으로 **Windows + (많은 경우) 32-bit Python/COM 환경** 제약이 걸립니다. 반면 현재 프로젝트에는 `venv64` 가 존재합니다. _snapshot_pyfiles _snapshot_tree

따라서 가장 안전한 운영 방식은:

- **(권장) 키움 수집은 별도 32-bit 실행기(별 venv)로 수행**
- 결과물은 **파일/DB로 저장**
- 메인(64-bit) 분석/백테스트는 저장된 결과물을 읽기만 함

즉, "수집기(32bit) ↔ 분석기(64bit)"를 **프로세스 경계로 분리**하면, 개발/운영이 훨씬 단단해집니다.

B. 기존 파이프라인을 "소스 교체형"으로 만들기

이미 가격 파이프라인 파일들이 분리돼 있습니다. _snapshot_pyfiles

- `src/collectors/kiwoom_prices.py` (키움 소스)
- `src/collectors/price/price_update_daily.py` (일별 업데이트 오케스트레이션)
- `src/collectors/price/price_backfill.py` (히스토리 백필)
- `src/collectors/price/price_store.py` (저장 스키마/저장소)

여기서 해야 할 일은 단순합니다.

1. **price_store는 그대로 유지** (스키마/저장 규격은 공용 표준)
2. `price_update_daily / backfill` 이 호출하는 "가격 데이터 공급자(provider)"를
   - (기존: 공공 데이터/라이브러리) → (개선: 키움) 으로 교체 또는 우선순위 1로 변경

이렇게 하면 레짐 전략이 요구하는 어떤 형태의 데이터라도 "항상 동일한 형식"으로 공급받을 수 있습니다.

C. 안정성을 위해 반드시 넣어야 하는 운영 로직(중요)

키움은 "가져오기만 하면 끝"이 아니라, 아래가 없으면 장기 운영 중 반드시 깨집니다.

- **증분 업데이트**: 종목별 "마지막 저장 일자" 이후만 요청

- **요청 스로틀/재시도**: 과도 호출 시 실패 → 지수 백오프 재시도
- **데이터 무결성 검사**:
  - 날짜 누락(거래일 갭),
  - 음수/0 가격,
  - OHLC 관계 위배(High < Low 등),
  - 거래량 비정상(연속 0 등)
    발견 시 재조회 또는 결측 처리
- **수정주가(액면분할/배당) 정책 통일**:
  레짐 분류/전이 추적에서는 "가격 연속성"이 핵심이라 **수정주가 기준으로 통일**하는 편이 일반적으로 유리합니다. (원주가 혼합은 전이 탐지에 치명적 노이즈를 넣습니다.)

---

3) 지금 단계에서의 "개발 진행 순서" 제안 (최단거리)

주가 수집이 불안정한 상태에서 레짐을 먼저 만들면, 뒤에서 계속 흔들립니다. 따라서 우선순위를 이렇게 두는 게 맞습니다.

Step 0. 현재 실패 원인을 "로그로 고정"

- 기존 `price_update_daily.py` 를 한 번 돌릴 때,
  - 어떤 종목에서,
  - 어떤 구간에서,
  - 어떤 예외로 멈추는지
    를 **로그로 남기고 재현** 가능하게 만드는 게 먼저입니다.

Step 1. 키움 수집 → price_store 저장까지 "단일 종목"으로 E2E 성공

- 삼성전자 1종목이라도 좋습니다.
- "키움으로 가져온 일봉이 price_store 규격으로 저장되고, 다시 로드해도 동일"이 확인되면 절반은 끝입니다.

Step 2. 200종목으로 확장 + 증분 업데이트 운영화

- "오늘자만 업데이트"가 매일 안정적으로 돌아가면,
- 과거 백필은 별도 작업(야간/주말)로 분리합니다.

Step 3. 그 다음에 regime 개발 착수

- 그때부터 `d:/quant/regime` 에서,
  - 레짐 분류(강상승/상승/보합/하락/강하락),
  - 기간별 상태,
  - 전이 매트릭스/체류일,
  - 이벤트 태그(3주/3일)
    를 올리면 됩니다.

---

4) 결론: "키움 활용"은 맞고, 구현은 '소스 교체 + 실행환경 분리'가 핵심입니다

정리하면,

- 키움 수집 코드가 이미 있고(`kiwoom_prices.py`), 가격 파이프라인도 분리돼 있으니
  (`price_store/update/backfill`), _snapshot_pyfiles
- **regime 폴더는 전략 개발용으로 두고**,
- **주가 수집은 기존 collectors/price 체계에 키움 소스를 붙여서 안정화**하는 것이 가장 안전합니다.

원하시면 다음 단계로, **(1) 현재 price_update_daily가 어디까지/왜 실패하는지 점검 포인트 체크리스트**와 **(2) 키움 수집 결과를 price_store로 적재하는 "표준 인터페이스(Provider/Adapter) 설계안"**을 바로 제시하겠습니다.

출처

d:/qunat 폴더에서 venv64 가상환경 활성화 하는 명령이 뭐였지?

`D:\Quant` 폴더에서 `venv64` 가상환경을 활성화하는 명령은 아래입니다. (PowerShell 기준)

powershell 코드 복사

```powershell
cd D:\Quant
.\venv64\Scripts\Activate.ps1
```

만약 PowerShell 실행 정책 때문에 막히면(보통 처음 1회), 현재 세션만 허용하는 방식으로:

powershell 코드 복사

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv64\Scripts\Activate.ps1
```

price_update_daily.py
Python

price.db
파일

지난번엔 prive_update_daily.py 실행하니까 에러가 났는데 오늘은 또 잘 되네? 파일을 실행했더니 db는 업데이트 된 거 같은데, 제대로 모든 종목이 업데이트 되진 않은 것 같아. 파일 내용과 db파일을 보고 유니버스 파일들의 과거 가격 데이터와 최근일 데이터까지 업데이트 된 건지 확인해줘.

2m 22s 동안 생각함 ›

확인해보니, **DB는 "업데이트가 된 것처럼 보이지만" 유니버스 전체가 최신까지 갱신된 상태는 아닙니다.** 원인은 `price_update_daily.py` 를 **인자 없이 실행하면 기본값으로 삼성전자(005930)만 업데이트**되기 때문입니다. price_update_daily

---

1) `price.db` (prices_daily) 실제 상태 요약

- 테이블: `prices_daily`
- 전체: **621개 티커**, **1,468,653 rows**
- DB 전체 최종일: **2026-01-26**

하지만 중요한 포인트:

- **2026-01-26 데이터가 존재하는 티커는 1개뿐**이고, 그 티커는 **005930(삼성전자)** 입니다.
- **570개 티커의 최종일이 2025-12-30**에서 멈춰 있습니다.
- 나머지 **50개 티커는 2025-12-30보다 더 과거에서 종료**되어 있습니다(상장폐지/코드변경/수집 누락 가능).

즉, 오늘 실행로 인해 DB의 `MAX(date)` 는 2026-01-26으로 보이지만, **대부분 종목은 2025-12-30 이후 데이터가 비어 있습니다.**

---

2) 왜 이런 현상이 생기나 (파일 내용 근거)

`price_update_daily.py` 는 기본값이:

- `--tickers` 기본값: **"005930"** price_update_daily
- `--tickers-file` 을 주지 않으면, 결국 **005930만 업데이트**합니다. price_update_daily

또 하나 중요한 점:

- DB에 티커가 **없으면** `start = end` 로 잡아서 **"오늘 하루만" 채웁니다**(과거 백필 없음). price_update_daily
  → 신규 편입 종목/누락 종목은 `price_backfill.py` 같은 별도 백필이 필요합니다.

---

3) 결론: "유니버스 파일의 과거~최신까지 업데이트"는 아직 아님

- 이번 실행은 **005930만 최신(2026-01-26)까지 갱신**된 것으로 보입니다.
- 유니버스 전체 갱신을 하려면, 실행 시 반드시 ** `--tickers-file` **로 유니버스 CSV를 넣고 돌려야 합니다. price_update_daily

예:

powershell 코드 복사

```powershell
cd D:\Quant
python price_update_daily.py --tickers-file data\universe.csv --ticker-col ticker
```

---

4) "진짜로 유니버스가 최신까지 갱신됐는지" 자동 검증 스크립트

아래 파이썬을 그대로 실행하면, **유니버스 종목별 최종일이 언제인지**와 **최신일 대비 뒤처진 종목 목록**을 바로 확인할 수 있습니다.

python 코드 복사

```python
import sqlite3
import pandas as pd
```

```python
DB_PATH = r"D:\Quant\price.db"
UNIV_CSV = r"D:\Quant\data\universe.csv"   # 경로 맞게 수정
TICKER_COL = "ticker"                      # 컬럼명 맞게 수정

# 1) 유니버스 로드
u = pd.read_csv(UNIV_CSV)
tickers = (
    u[TICKER_COL].astype(str).str.strip().str.replace(r"\.0$", "", regex=True)
)
tickers = [t.zfill(6) for t in tickers if t and t != "nan"]
tickers_set = set(tickers)

# 2) DB에서 유니버스 티커의 최종일 조회
con = sqlite3.connect(DB_PATH)
q = f"""
SELECT ticker, MIN(date) AS min_date, MAX(date) AS max_date, COUNT(*) AS n
FROM prices_daily
WHERE ticker IN ({",".join(["?"]*len(tickers_set))})
GROUP BY ticker
"""
df = pd.read_sql_query(q, con, params=list(tickers_set))
con.close()

# 3) DB에 없는 티커(수집 자체가 안 된 종목)
in_db = set(df["ticker"].tolist())
missing = sorted(list(tickers_set - in_db))

# 4) 최신일 기준으로 뒤처진 티커
df["max_date"] = pd.to_datetime(df["max_date"])
latest = df["max_date"].max()
lagging = df[df["max_date"] < latest].sort_values("max_date")

print("유니버스 티커 수:", len(tickers_set))
print("DB에 존재하는 유니버스 티커 수:", len(in_db))
print("DB에 없는 유니버스 티커 수:", len(missing))
print("유니버스 기준 최신일:", latest.date())

print("\n[DB에 없는 티커(상위 20개)]")
print(missing[:20])

print("\n[최신일 대비 뒤처진 티커(상위 30개)]")
print(lagging[["ticker","max_date","n"]].head(30))
```

---

5) 다음 조치(우선순위)

1. **유니버스 CSV를 넣어서** `price_update_daily.py` **를 다시 실행**
2. 위 검증 스크립트로 **"최신일이 유니버스 대부분에 적용됐는지"** 확인
3. 신규/누락 종목(히스토리 짧음)은 `price_backfill.py` 로 **과거 데이터 백필**
   - 레짐(252/126/63) 계산을 하려면 **최소 252거래일 이상**이 필요하므로, `n < 252` 종목은 레짐 계산에서 제외하거나(초기) 별도 처리해야 합니다.

원하시면, 다음 단계로 **(1)** `price_update_daily.py` **가 실제로 어떤 유니버스 파일을 쓰도록 운영하는 게 맞는지**(Top200 월별 유니버스 vs 고정 유니버스), 그리고 **(2) 백필/일별 업데이트를 분리 운영하는 표준 실행 스크립트**까지 정리해 드리겠습니다.

출처

Universe Union Kospi Top200 201501 202512

	ticker
1	000030
2	000040
3	000050
4	000060
5	000070
6	000080

Universe Krx All 20251230

	ticker	name	market
1	060310	3S	
2	095570	AJ네트웍스	
3	006840	AK홀딩스	
4	054620	APS	
5	265520	AP시스템	
6	211270	AP위성	

Universe Krx Kospi 20251230

	ticker	name	market
1	095570	AJ네트웍스	KOSPI
2	006840	AK홀딩스	KOSPI
3	027410	BGF	KOSPI
4	282330	BGF리테일	KOSPI
5	138930	BNK금융지주	KOSPI
6	001460	BYC	KOSPI

Universe Monthly Kospi Top200 201501 202512

	asof	used_date	ticker	name
1	20150131	20150131	096760	JW홀딩스
2	20150131	20150131	003280	흥아해운
3	20150131	20150131	068400	SK렌터카
4	20150131	20150131	006840	AK홀딩스
5	20150131	20150131	027410	BGF
6	20150131	20150131	138930	BNK금융지주

Universe Top200 Kospi 20251230

	ticker	name	시가총액
1	005930	삼성전자	709764586847800
2	000660	SK하이닉스	473929539615000
3	373220	LG에너지솔루션	86229000000000
4	207940	삼성바이오로직스	78463161945000
5	005935	삼성전자우	72784940028800
6	005380	현대차	60710677619000

지금 내 피씨에 있는 universe 파일들이야. 어떤 게 최종적이고 지금 작업에 적합한지 확인해 줘.

1m 4s 동안 생각함 ›

올려주신 5개 universe 파일을 구조/내용 기준으로 보면, **"최종(주력)로 써야 하는 파일"은 1개가 아니라 목적별로 역할이 다릅니다.** 다만 지금 하시려는 작업(Top200 기반 레짐/전이 + 가격 업데이트 안정화)에는 **딱 2개가 핵심**이고, 나머지는 보조입니다.

아래는 파일별 성격과 "지금 작업에 적합한지" 판단입니다.

---

1) 파일별 용도 판정

A. `universe_monthly_kospi_top200_201501_202512.csv` ✅ **"전략/백테스트의 기준(정본)"**

- **형태**: (asof, used_date, ticker, name) / 2015-01 ~ 2025-12 **매월 200종목** (총 132개월 × 200 = 26,400 행)
- **의미**: "월별로 Top200 구성 종목이 어떻게 바뀌었는지"가 들어있는 **동적 유니버스의 원본**
- **결론**: 레짐 히스토리/전이 분석을 "유니버스 변화까지 반영"하려면 이 파일이 **최종 기준**입니다.

B. `universe_union_kospi_top200_201501_202512.csv` ✅ **"가격수집/백필의 기준(정본)"**

- **형태**: ticker 619개(유니크)
- **의미**: 위 월별 Top200에 한 번이라도 등장한 종목의 **전체 합집합**
- **결론**: 가격 DB를 안정적으로 만들려면, **백필(backfill)은 이 파일이 최적**입니다.
  (월별 파일을 그대로 tickers로 쓰면 중복이 26,400행이라 비효율적입니다.)

  참고: 월별 파일의 고유 ticker 수가 619개이고, union 파일도 619개라서, union은 월별의 "정리본"입니다.

C. `universe_top200_kospi_20251230.csv` ✅ **"현재 Top200(스냅샷)"**

- **형태**: ticker 200개 + name + 시가총액
- **의미**: 2025-12-30 기준 Top200 **정적 리스트**
- **결론**: "오늘부터 운용/점검은 최신 Top200만 업데이트" 같은 목적이면 이 파일이 적합합니다.
  단, 날짜가 2025-12-30이라 **2026년으로 넘어간 현재 시점(2026-01-26)에서는 재생성 필요**합니다.

D. `universe_krx_kospi_20251230.csv` △ **"참고용(코스피 전체 스냅샷)"**

- **형태**: ticker 958개 + name + market(KOSPI)
- **의미**: 코스피 전체 목록(참고/매핑/검증용)
- **결론**: Top200 전략의 유니버스 파일로 쓰기보다는, **종목 마스터 검증/보완** 용도에 가깝습니다.

E. `universe_krx_all_20251230.csv` △ **"참고용(전체 상장 스냅샷)"**

- **형태**: ticker 2,903개 + name + market(현재 파일은 market 값이 비어 있음)
- **결론**: 전체 종목 풀을 만들거나 검증할 때 참고는 되지만, 현 상태로는 market 컬럼이 비어 있어 **실전 유니버스 기준으로는 부적합**합니다.

---

2) "지금 작업"에 최적 조합(권장)

선생님 목표(레짐/전이 전략 + 가격 수집 안정화) 기준으로는 아래처럼 쓰는 게 가장 깔끔합니다.

1. **동적 유니버스/백테스트의 기준(정본)**
   → `universe_monthly_kospi_top200_201501_202512.csv`
2. **가격 DB 백필/누락 점검의 기준(정본)**
   → `universe_union_kospi_top200_201501_202512.csv` (619개)
3. **일별 업데이트(운영) 기준(선택)**
   - "Top200만 매일 업데이트"면 → `universe_top200_kospi_20251230.csv`
   - "동적 유니버스에 등장했던 619개를 모두 최신까지 유지"면 → `universe_union...csv`

---

3) 실무적으로 중요한 주의점(가격 업데이트 관련)

`price_update_daily.py` 는 **tickers-file을 무엇으로 주느냐에 따라 결과가 완전히 달라집니다.**

- **월별 파일(** `universe_monthly...` **)을 그대로 넣으면** ticker가 26,400행(중복 포함)이라 업데이트가 불필요하게 느려지고, 같은 종목을 반복 호출할 가능성이 큽니다.
- 따라서 업데이트 대상은 `union` **(619) 또는 최신** `top200` **(200)** 둘 중 하나로 주는 것이 맞습니다.

결론

- **최종(정본) 유니버스**: `universe_monthly_kospi_top200_201501_202512.csv`
- **가격 수집/백필용 최종 리스트**: `universe_union_kospi_top200_201501_202512.csv`
- **운영용(현재 Top200만)**: `universe_top200_kospi_20251230.csv` (단, 날짜 갱신 필요)

원하시면 다음 단계로, 지금 `price.db` 기준에서

- **union 619개 중 "최신일(예: 2026-01-26)까지 갱신된 종목 비율"**
- **유니버스(월별/Top200) 기준으로 누락된 종목 목록**
  을 바로 뽑아서 "왜 일부만 업데이트됐는지"까지 원인분류해드리겠습니다. (지금은 파일 기준 판단까지 완료했습니다.)

그런거 체크하지 말고, 그냥, 업데이트 하면 안돼? 체크가 왜 필요하지?

업데이트 "만" 할 수도 있습니다. 다만 **지금 상황(겉으로는 DB가 최신처럼 보이는데 실제로는 일부만 갱신되는 상태)**에서는 체크 없이 계속 업데이트를 돌리면, 나중에 레짐 계산/백테스트 단계에서 **조용히 깨진 결과(결측·편향·전이 오류)**가 나와서 원인 추적 비용이 훨씬 커집니다. 체크는 "품질관리"라기보다 **업데이트가 실제로 제대로 됐는지 확인하는 최소 안전장치**입니다.

왜 필요한지, 핵심만 말씀드리겠습니다.

---

1) "업데이트 성공"과 "유니버스가 최신"은 다릅니다

지금 DB는 `MAX(date)` 가 최신으로 찍혀도, **그 날짜가 1개 종목(예: 005930)에만 존재하면** DB는 최신처럼 보입니다.
즉, 업데이트가 "완료" 메시지를 내도 **유니버스의 대부분이 낙오**할 수 있습니다. (이미 그 징후가 있었습니다.)

---

2) 체크 없이 가면 생기는 실제 문제(레짐 전략에 치명적)

레짐은 63/126/252일 구간을 쓰므로, 데이터가 조금만 끊겨도 다음 문제가 생깁니다.

- 종목별 최신일이 다르면 **같은 날짜에 레짐 비교가 불가능**
- 일부 종목만 최신이면 "전이 이벤트"가 **거짓으로 발생/소멸**
- 계산 시 결측을 자동으로 drop하면 **유니버스가 몰래 줄어듦**
- 백테스트가 돌아가긴 돌아가지만, 결과는 **편향된 샘플**로 나옵니다(가장 위험)

이건 "정확도"가 아니라 **시스템 신뢰성** 문제입니다.

---

3) 체크는 거창할 필요가 없습니다: 10초짜리 2개만 하면 됩니다

선생님이 원하시는 "그냥 업데이트"를 하되, 최소한 아래 2개만 자동으로 찍으면 충분합니다.

1. **업데이트 대상 티커 수 vs 성공한 티커 수**
2. **최신일 미달 티커 목록(Top 20)**

이 두 줄만 있으면 "오늘 업데이트가 정상인지"를 바로 압니다.

---

4) 그럼 "체크 안 하고 그냥 업데이트"를 하려면 조건이 있습니다

체크 없이도 안전하려면 업데이트 로직 자체가 아래를 보장해야 합니다.

- 대상 티커를 **반드시 tickers-file(Union 619 또는 Top200 200)**로 넣는다
- 각 티커별로 **마지막 날짜 이후를 증분 업데이트**
- 실패 종목은 **재시도 후에도 실패하면 실패 목록을 남긴다**
- 실행 마지막에 **성공/실패 요약을 출력한다** (이게 사실상 체크입니다)

즉, 체크를 "안 하는 것"이 아니라 **업데이트 과정에 체크를 내장**시키는 게 맞습니다.

---

5) 실행 관점에서 가장 단순한 해법

선생님이 원하는 방향("그냥 업데이트")을 최대한 존중하면서 현실적으로 안전한 방식은:

- 업데이트 대상은 ** `universe_union_kospi_top200_201501_202512.csv` (619개)**로 고정
- `price_update_daily.py` 를 그 파일로 실행
- 실행 종료 시, **최신일 미달 종목만** 간단히 출력

이건 "분석용 체크"가 아니라, **업데이트 실패를 바로 알기 위한 운영 로그**입니다.

원하시면 제가 바로 다음을 만들어 드리겠습니다.

- `D:\Quant\price_update_union.ps1` : union 619개를 대상으로 업데이트 실행
- `price_update_daily.py` 에 "성공/실패 요약 + 최신일 미달 티커 출력"을 최소한으로 추가(기존 구조 유지)

그러면 선생님이 원하시는 "그냥 업데이트"가 실제로 가능한 운영 형태가 됩니다.

알았어. 그럼, 먼저 왜 일부만 업데이트 된건지, 우리가 안정적으로 데이터를 수집하려면 어떻게 하면 되는지 분석해 줘.

1m 37s 동안 생각함 ›

분석해보면 "일부만 업데이트된" 이유는 기술적으로 명확합니다. 결론부터 말씀드리면, **업데이트가 일부만 된 게 아니라, 사실상 005930(삼성전자)만 최신까지 갱신된 상태**입니다. DB의 `MAX(date)` 가 최신으로 찍히는 바람에 "전체가 최신까지 된 것처럼 보였던" 케이스입니다.

---

1) 왜 일부만 업데이트됐나: 원인 1순위는 실행 방식입니다

현재 `price_update_daily.py` 는 인자를 주지 않고 실행하면 기본값이 이렇게 되어 있습니다.

- `--tickers` 기본값: **"005930"**
- `--tickers-file` 이 없으면 tickers 문자열만 사용

즉, 선생님이 파일을 "그냥 실행"하시면 **005930만 업데이트**되는 구조입니다.
그래서 DB에서 "최신일(예: 2026-01-26)" 데이터가 005930에만 생기고, 나머지는 2025-12-30에서 멈춰 있게 됩니다.

추가로 중요한 포인트:

- DB에 티커가 "처음 등장"하는 경우, 이 코드가 **과거 백필을 하지 않고** `start=end` **로 '오늘 하루만' 채웁니다.**
  - 즉, 새로운 종목/누락 종목은 업데이트를 돌려도 과거가 안 채워집니다(별도 backfill 필요).

---

2) 지난번엔 에러, 오늘은 성공: 왜 그럴 수 있나

이 스크립트는 내부적으로 `fetch_ohlcv_pykrx()` (pykrx 기반)를 호출합니다. 이 계열은 다음 상황에서 **간헐 오류**가 생기기 쉽습니다.

- 네트워크/서버 응답 불안정(일시적인 실패)
- 요청량이 많을 때(특히 여러 종목 연속 호출)
- 특정 종목이 pykrx에서 일시적으로 조회 실패(공휴일/데이터 공백/종목 상태 영향)
- `end=date.today()` 가 "거래일이 아닌 날"일 때(결과가 빈 DF이거나 예외 처리에 따라 실패)

즉, "오늘은 잘 됐다"는 건 **소스가 안정적이라는 의미가 아니라**, 운 좋게 성공했을 가능성이 큽니다. 그래서 운영 관점에서는 구조를 바꾸는 게 맞습니다.

---

3) 안정적으로 데이터를 수집하려면: 운영 구조를 이렇게 나누는 게 정답입니다

선생님 목표(장기적으로 안정적 수집 + 레짐/전략 개발) 기준으로, 아래처럼 **업데이트 작업을 2종으로 분리**하면 거의 깨지지 않습니다.

A) 과거 백필(backfill) 작업: "합집합 619개"로 '한 번' 크게 채우기

- 대상: `universe_union_kospi_top200_201501_202512.csv` (619개)
- 목적: 레짐 계산에 필요한 최소 252거래일 이상 확보
- 빈도: 1회 또는 누락 종목에 대해서만 추가

B) 매일 증분 업데이트(daily update): "현재 운용 대상"만 매일 갱신

- 대상 선택지(둘 중 하나)
  1. **현재 Top200(200개)**: 운용/점검이 가장 가볍고 안정적
  2. **union 619개 전체**: 리서치/유니버스 변화를 계속 추적하려는 경우(부하 큼)
- 빈도: 매일(장 마감 후)

  핵심은 "매일 619개를 다 돌리면" 실패 확률이 급격히 올라갑니다.
  운영은 200개(또는 더 줄인 watchlist), 리서치는 619개를 주 1회/월 1회 등으로 분리하는 게 장기적으로 안정적입니다.

---

4) 키움(OpenAPI)로 안정화하려면: '수집 프로세스 분리'가 핵심입니다

선생님이 말씀하신 것처럼, 키움은 장기 운영에 유리할 수 있습니다. 다만 실전에서는 다음 구조가 가장 안정적입니다.

- **키움 수집기(32-bit 환경/별도 venv)**
  → `price.db` 에 적재(또는 parquet로 떨어뜨린 뒤 적재)
- **메인 분석/백테스트(64-bit venv64)**
  → DB에서 읽기만 함

이렇게 하면:

- 키움 제약(32bit/COM/HTS 로그인 등)이 메인 분석 환경을 오염시키지 않고,
- 장애가 나도 "수집기만" 재시작하면 되고,
- 레짐/전략 개발은 venv64에서 계속 진행 가능합니다.

---

5) "안정 수집"을 위해 코드에 반드시 들어가야 하는 최소 기능(체크가 아니라 운영 안전장치)

선생님이 원하시는 건 "매번 복잡한 체크"가 아니라 "그냥 잘 돌게" 하는 겁니다. 그걸 가능하게 하는 최소 기능은 아래 3개입니다.

1. **대상 티커를 항상 파일로 받기(기본값 005930 제거 또는 경고)**
   - 실수로 1개만 업데이트되는 상황을 원천 차단
2. **실패 티커 리스트를 파일로 저장하고 재시도 큐로 돌리기**
   - 예: `logs/price_update_fail_YYYYMMDD.txt`
3. **end를 '오늘'이 아니라 '마지막 거래일'로 자동 정하기**
   - 가장 간단한 방법: 005930 같은 대표 종목으로 최신 거래일을 먼저 한번 조회해 end로 사용

이 3개만 있어도 "매일 돌려도 대부분 깨지지 않는 파이프라인"이 됩니다.

---

6) 지금 당장 해야 할 조치(원인 해결 관점)

"일부만 업데이트" 문제의 직접 해결은 간단합니다.

- 앞으로는 반드시 `--tickers-file` 을 주고 실행해야 합니다.
  - 예: **union 619개를 최신까지 맞추려면**

    powershell 코드 복사

    ```powershell
    python price_update_daily.py --tickers-file data\universe_union_kospi_top200_201501_20251
    ```

  - 예: **현재 Top200만 매일 업데이트하려면**

    powershell 코드 복사

    ```powershell
    python price_update_daily.py --tickers-file data\universe_top200_kospi_YYYYMMDD.csv --tic
    ```

그리고 "키움으로 바꾸겠다"면, 위 daily update 자리에 **키움 수집기를 끼워 넣는 구조**로 가면 됩니다(기존 price_store/DB 스키마는 유지).

다음 단계 제안(개발 착수용, 선택 없이 제가 바로 진행 가능)

1. `price_update_daily.py` 를 **운영형으로 개조**
   - 기본값 005930 제거/경고
   - 실패 목록 저장
   - end=마지막 거래일 자동화
   - 종목별 저장 row=0이 연속될 때 원인 기록
2. (선택) 키움 수집 모듈을 **price_store에 어댑터 형태로 붙이기**
   - 32bit 수집기 → 동일 DB로 적재

원하시면, 지금 업로드된 `price_update_daily.py` 기준으로 **수정 버전(새 파일) 스펙과 코드 뼈대**를 바로 작성해서 `D:\Quant` 에 붙여 쓸 수 있게 정리해 드리겠습니다.

일단 기존의 수집방법이 되니까 이걸로 먼저 619개 종목 데이터를 수집해서 마스터 db를 만들고, top200개 종목은 매일 주가 데이터를 업데이트 하는 게 좋겠어. 그런데, 처음에 수집해서 마스터 db를 만들때 252일 거래일로 제한하지 말고 서버 쪽에서 제공하는 최대한의 날짜를 가져오는 건 어떨까? 백테스트를 하기 위해서.

12s 동안 생각함 ›

네, **마스터 DB를 만들 때 252거래일로 제한하지 않고 "가능한 최대 히스토리"를 가져오는 방향이 더 낫습니다.** 특히 선생님처럼 백테스트/전략 검증을 염두에 두면, 초기에 데이터베이스를 넓게 깔아두는 게 이후 시행착오 비용을 줄입니다.

다만 "무제한으로 최대"를 하려면, 운영상 몇 가지를 같이 설계해야 안정적으로 끝까지 갑니다.

---

1) 마스터 DB는 "최대한 길게"가 맞는 이유

- **백테스트 기간을 길게** 잡을 수 있습니다(국면 다양성 확보: 상승/하락/금리변동/코로나 등).
- 레짐/전이 분석에서 **체류기간/전이확률이 더 안정적으로 추정**됩니다.
- 전략 개발 시 **훈련/검증(예: 2015~2020 vs 2021~2025)** 같은 분리가 가능해집니다.
- 나중에 "더 길게 다시 받자"가 되면, 시간이 훨씬 더 듭니다(이미 DB 스키마/파이프라인이 굳어져서).

결론: **초기 구축은 길게**, 운영 업데이트는 **좁게**가 정석입니다.

---

2) 하지만 "최대한 길게"는 이렇게 해야 안정적입니다 (필수 운영 설계)

최대 히스토리를 받으려면 619개 종목에 대해 호출량/실패율이 늘어납니다. 그래서 아래 4가지는 사실상 필수입니다.

1. **백필(backfill)과 데일리 업데이트를 완전히 분리**
- 백필: 오래 걸리는 작업(중단/재개 가능해야 함)
- 데일리: 짧고 확실한 작업(매일 자동 실행 가능)
2. **재시도 + 실패 목록 저장**
- 실패한 티커만 모아서 재실행할 수 있어야 "끝까지" 갑니다.
3. **증분 적재(Upsert)**
- "다시 돌려도 이어서 채워지는" 구조여야 합니다.
4. **마지막 거래일 기준으로 end를 고정**
- 오늘이 거래일이 아니면 데이터가 비거나 흔들립니다.
- 매 실행 시 "최신 거래일"을 먼저 확정하고 그 날짜로 end를 고정하는 방식이 안정적입니다.

3) 권장 운영안 (선생님이 제시하신 방향을 그대로 체계화)

A. 1회성/주기성 "마스터 백필"

- 대상: `universe_union_kospi_top200_201501_202512.csv` (619개)
- 기간: **서버가 제공하는 가능한 최대로**
  - 구현에서는 보통 `start='19900101'` 같은 아주 이른 날짜를 넣고,
  - 실제로는 "상장일 이후부터" 데이터가 내려오므로 자동으로 적정 범위가 됩니다.
- 산출: `price.db` 의 `prices_daily` 에 전 종목 장기 히스토리 적재

  여기서 252일 제한은 "레짐 계산 최소 요건"일 뿐, 마스터 구축 제한으로 쓰면 손해입니다.

B. 매일 "Top200만 업데이트"

- 대상: `universe_top200_kospi_YYYYMMDD.csv` (최신일자로 재생성 권장)
- 기간: 각 티커의 마지막 저장일 다음날~최신 거래일까지 증분
- 목적: "오늘 기준 운용/분석 대상"만 최신 유지

---

4) 현실적인 트레이드오프(알고 진행하시면 됩니다)

최대 히스토리 백필은 장점이 크지만, 비용도 있습니다.

- **시간/실패율 증가**: 오래된 구간일수록 결측/변동 이벤트가 많아 실패 가능성이 올라갑니다.
- **DB 용량 증가**: 619개 × 수천 거래일이면 수백만~천만 행 수준까지 갈 수 있습니다.
- **정합성 이슈 가능성**: 액면분할/합병/상장폐지/거래정지 등의 이벤트가 길게 잡힐수록 늘어납니다.

그래서 저는 "최대 히스토리"에 찬성하되, 반드시 위의 운영 설계를 같이 넣는 것을 전제로 추천드립니다.

---

5) 실행 관점에서의 구체적 권장(바로 적용 가능한 수준)

- **백필은 union(619)으로**, start는 아주 과거로 크게 잡되,
  - 종목별로 "이미 DB에 있는 최소일(min_date)"을 확인해서 그 이전만 받거나,
  - 또는 단순히 start를 고정하되, **DB upsert로 중복 적재가 안 되게** 처리합니다.
- **데일리는 top200(200)로**, 각 티커별 max_date 다음날부터만 받습니다.

원하시면 다음 단계로, 지금 프로젝트에 이미 있는 `price_backfill.py` / `price_store.py` / `price_update_daily.py` 흐름을 기준으로:

1. **union 619 백필을 "최대 히스토리"로 수행하는 실행 커맨드/옵션 설계**
2. **실패 티커 재시도 큐 + 중단/재개 가능한 백필 러너**
3. **top200 데일리 업데이트를 안정적으로 매일 돌리는 운영 스크립트**

이 3가지를 "실행 스크립트 기준"으로 딱 만들어 드리겠습니다.

다만 여기서 하나만 확인하면 좋습니다(결정이 아니라 정보 확인):
현재 `price_backfill.py` 가 start 날짜를 인자로 받는 구조인지, 아니면 내부에 고정되어 있는지입니다. 파일을 올려주시면, 그 파일 기준으로 "최대 히스토리 백필"로 바로 개조해 드리겠습니다.

내가 준 기존 파일 리스트에 backfill 이 된 시장가격 수집 db 파일은 없는 건가?

35s 동안 생각함 ›

있습니다.

선생님이 올려주신 **폴더 트리 스냅샷(_snapshot_tree.txt)** 기준으로, 프로젝트 안에 DB 파일은 크게 2개만 보이고 그중 **시장가격(주가) 수집용 DB는** `price.db` **가 맞습니다.**

- 위치: `D:\Quant\data\db\price.db`
- 같은 폴더에 DART용 DB도 있습니다: `D:\Quant\data\db\dart_main.db`

즉, "backfill(과거 데이터 누적)된 시장가격 수집 DB 파일"로 추정되는 건 `price.db` **1개**입니다.
(추가로 별도의 `prices_*.db` 같은 다른 가격 DB 파일은 스냅샷에는 없습니다.)

원하시면 다음 단계로, ** `price.db` 가 '백필이 어느 범위까지' 실제로 들어가 있는지(티커별 시작일/최종일 분포)**를 짧은 쿼리로 한 번에 확인할 수 있게 점검 스크립트만 드리겠습니다.

아래 스크립트로 `price.db` 의 **백필 범위(종목별 시작일/최종일/행수)**와 **전체 커버리지 분포**를 한 번에 확인할 수 있습니다.

- 기본 DB 경로: `D:\Quant\data\db\price.db`
- 테이블명은 현재 사용 중인 `prices_daily` 로 가정했습니다. (다르면 스크립트에서 `TABLE` 만 바꾸시면 됩니다.)

---

1) 빠른 점검 스크립트 (전체 티커 기준)

python 코드 복사

```python
# check_price_db_coverage.py
import sqlite3
import pandas as pd

DB_PATH = r"D:\Quant\data\db\price.db"
TABLE = "prices_daily"   # 다르면 수정

con = sqlite3.connect(DB_PATH)

# 1) 테이블 존재 확인
tables = pd.read_sql_query(
    "SELECT name FROM sqlite_master WHERE type='table' ORDER BY name;", con
)
print("Tables:", tables["name"].tolist())

# 2) 종목별 min/max/count
q = f"""
SELECT
  ticker,
  MIN(date) AS min_date,
  MAX(date) AS max_date,
  COUNT(*)  AS n_rows
FROM {TABLE}
GROUP BY ticker
ORDER BY max_date DESC, n_rows DESC
"""
df = pd.read_sql_query(q, con)
con.close()

# 날짜 파싱
df["min_date"] = pd.to_datetime(df["min_date"], errors="coerce")
df["max_date"] = pd.to_datetime(df["max_date"], errors="coerce")

print("\n[Summary]")
print("tickers:", len(df))
print("rows:", int(df["n_rows"].sum()))
latest = df["max_date"].max()
earliest = df["min_date"].min()
print("global earliest:", earliest.date() if pd.notna(earliest) else None)
print("global latest  :", latest.date() if pd.notna(latest) else None)

# 3) 최신일 기준으로 뒤처진 종목 상위
lagging = df[df["max_date"] < latest].sort_values("max_date")
print("\n[Lagging tickers (top 30)]")
print(lagging[["ticker", "max_date", "n_rows"]].head(30).to_string(index=False))

# 4) 히스토리 길이(행수) 부족 종목: 레짐 계산 최소 252일 기준
short = df[df["n_rows"] < 252].sort_values("n_rows")
print("\n[Short history (<252 rows) tickers (top 30)]")
print(short[["ticker", "min_date", "max_date", "n_rows"]].head(30).to_string(index=False))

# 5) 분포 요약(최종일, 히스토리 길이)
print("\n[Max_date distribution (top 15)]")
print(df["max_date"].dt.date.value_counts().head(15).to_string())

bins = [0, 50, 100, 252, 500, 1000, 2000, 5000, 10**9]
labels = ["<50", "50-99", "100-251", "252-499", "500-999", "1000-1999", "2000-4999", ">=5000"]
df["len_bucket"] = pd.cut(df["n_rows"], bins=bins, labels=labels, right=False)
print("\n[History length buckets]")
print(df["len_bucket"].value_counts().sort_index().to_string())

# 6) CSV로 저장(원하면)
```

```
out = r"D:\Quant\reports\price_db_coverage.csv"
try:
    df.sort_values(["max_date", "n_rows"], ascending=[False, False]).to_csv(out, index=False, enco
    print(f"\nSaved: {out}")
except Exception as e:
    print(f"\nCould not save CSV to {out}: {e}")
```

실행 방법 (venv64)

powershell 코드 복사

```powershell
cd D:\Quant
.\venv64\Scripts\Activate.ps1
python check_price_db_coverage.py
```

---

2) 유니버스(619 or Top200)만 따로 점검하고 싶으면 (옵션)

위 스크립트에 아래 블록을 추가해서 "유니버스 티커만 필터"해서 보실 수 있습니다.

python 코드 복사

```python
# 유니버스 파일을 쓰는 경우(예: union 619)
UNIV_CSV = r"D:\Quant\data\universe_union_kospi_top200_201501_202512.csv"
TICKER_COL = "ticker"

u = pd.read_csv(UNIV_CSV)
tickers = u[TICKER_COL].astype(str).str.strip().str.replace(r"\.0$", "", regex=True)
tickers = [t.zfill(6) for t in tickers if t and t != "nan"]

df_u = df[df["ticker"].isin(tickers)].copy()
print("\n[Universe-only summary]")
print("universe tickers:", len(tickers))
print("in DB:", df_u["ticker"].nunique())
```

원하시면, 위 출력 결과(특히 `Lagging tickers`, `Short history`)를 붙여주시면 "왜 특정 날짜에서 멈추는지"를 케이스별로 바로 분류해서 `price_update_daily.py / backfill` 운영 규칙까지 확정해 드리겠습니다.

이렇게 나오는데?
'''
(venv64) PS D:₩Quant> python check_price_db_coverage.py
Tables: ['prices_daily']

[Summary]
tickers: 621
rows: 1468653
global earliest: 2013-10-14
global latest : 2026-01-26

[Lagging tickers (top 30)]

ticker	max_date	n_rows
000800	2015-04-14	70
192520	2015-06-22	116
010520	2015-07-14	132
003605	2015-08-13	154
003600	2015-08-13	154
000830	2015-09-14	175
000687	2015-11-25	224
025850	2016-03-14	295
008020	2016-05-18	339
003450	2016-10-31	451
011720	2016-11-18	465
068870	2017-01-16	505
068875	2017-01-16	505
037620	2017-01-19	508
117930	2017-03-06	537
110570	2017-04-14	566
001689	2017-04-27	575
021960	2017-07-20	630
002550	2017-07-20	630
005255	2017-08-04	641
000327	2017-09-27	678
008000	2018-03-19	789

```
192530 2018-10-25      936
002000 2018-12-12      970
004135 2018-12-18      974
004130 2018-12-18      974
000030 2019-02-12     1008
002250 2019-05-17     1073
079440 2020-02-13      676
027390 2020-04-14     1299

[Short history (<252 rows) tickers (top 30)]
ticker    min_date    max_date  n_rows
0126Z0 2025-11-24 2025-12-30      26
000800 2015-01-02 2015-04-14      70
439260 2025-08-01 2025-12-30     101
192520 2015-01-02 2015-06-22     116
010520 2015-01-02 2015-07-14     132
483650 2025-05-22 2025-12-30     150
003605 2015-01-02 2015-08-13     154
003600 2015-01-02 2015-08-13     154
000830 2015-01-02 2015-09-14     175
031210 2025-03-14 2025-12-30     196
064400 2025-02-05 2025-12-30     222
000687 2015-01-02 2015-11-25     224
499790 2024-12-23 2025-12-30     247

[Max_date distribution (top 15)]
max_date
2025-12-30    570
2023-07-14      4
2017-01-16      2
2015-08-13      2
2018-12-18      2
2017-07-20      2
2025-07-14      1
2024-07-08      1
2024-12-06      1
2025-07-30      1
2026-01-26      1
2025-11-20      1
2025-12-12      1
2023-02-20      1
2023-04-24      1

[History length buckets]
len_bucket
<50            1
50-99          1
100-251       11
252-499       14
500-999       24
1000-1999     60
2000-4999    510
>=5000         0

Saved: D:₩Quant₩reports₩price_db_coverage.csv
(venv64) PS D:₩Quant>
```

출력 결과가 말해주는 건 명확합니다. **현재 `price.db` 는 "과거 백필은 상당히 잘 되어 있는데(대부분 2,000~4,999행), 최신 업데이트가 유니버스 전체에 적용되지 않았습니다."** 그리고 "왜 일부만 업데이트됐는지"도 데이터 패턴만 봐도 거의 확정할 수 있습니다.

아래처럼 해석하시면 됩니다.

---

1) 현재 DB 상태 진단(핵심)

A. 과거 데이터(백필)는 대체로 충분

- `global earliest: 2013-10-14`
- `len_bucket` 에서 **2000~4999 구간이 510개** → 대부분 종목은 장기간 일봉이 있습니다.
- 즉 "252일 제한 없이 최대한 가져오기" 관점에서, 이미 마스터 DB의 베이스는 충분히 구축돼 있습니다.

B. 최신 데이터는 사실상 "멈춘 상태"

- `max_date distribution` 에서 **2025-12-30이 570개**
  → **570개 종목이 2025-12-30에서 일괄적으로 멈춤**
- **2026-01-26은 1개 종목만 존재**
  → 오늘 실행으로 특정 종목(아마 005930)이 최신까지 갱신된 형태

즉, "일부만 업데이트"가 아니라, **대다수는 업데이트가 전혀 진행되지 않았고** 005930만 최신으로 간 케이스에 가깝습니다.

---

2) 왜 570개가 2025-12-30에서 멈췄나 (가능성 높은 원인 2가지)

원인 1) 업데이트가 그 이후 종목들에 대해 실행되지 않음

가장 가능성이 큽니다.

- `price_update_daily.py` 를 인자 없이 실행하면 기본이 **005930만 업데이트**라서, 실제로는 대부분 종목이 갱신되지 않습니다.
- 그 결과 "최신일이 있는 종목만 최신처럼 보이는" 상태가 됩니다.

원인 2) 2025-12-30 이후 구간 조회가 대량 실패(소스/환경 문제)

만약 tickers-file로 여러 종목을 돌렸는데도 2025-12-30에서 멈췄다면,

- **데이터 소스(pykrx) 측 장애/차단/응답 변경**
- 또는 "end 날짜 처리"가 거래일 아닌 날짜와 충돌
- 또는 네트워크/요청 제한으로 다수 실패 후 조기 종료
  같은 운영 장애 가능성이 있습니다.

다만 현재 분포가 "570개가 동일 날짜"로 멈춘 점을 보면, **구조적으로 '그날까지 한번 백필된 이후, 그 이후 업데이트가 제대로 돌지 않았다'**는 쪽이 더 설득력 있습니다.

---

3) "Lagging tickers" 목록이 의미하는 것

여기 보이는 종목들은 2015~2020 등에서 끝나 있습니다. 이는 보통 아래 케이스입니다.

- **상장폐지/합병/종목코드 변경**
- **우선주/리츠/특수코드**(예: `003600/003605`, `068870/068875`, `004130/004135` 처럼 쌍으로 존재)
- **신규상장**이라 데이터 길이가 짧음(예: 2025년 상장 종목들)

이건 "업데이트 실패"라기보다 **종목 이벤트/정상 종료**일 가능성이 큽니다.
즉, 레짐 계산에서는 **현재 거래 중인 종목만 필터링**하면 됩니다.

---

4) 지금 당장 해야 할 "안정화" 조치(운영 설계)

선생님이 원하신 운영 방식은 매우 좋습니다.

- **마스터 DB**: union 619개를 가능한 길게 확보(이미 상당 부분 되어 있음)
- **데일리 업데이트**: top200만 매일 최신까지

여기서 "안정화"의 핵심은 **데일리 업데이트가 확실히 200개 전부를 갱신하도록 만드는 것**입니다.

운영 원칙(필수)

1. `price_update_daily.py` 는 **항상** `--tickers-file` **로 실행** (Top200 파일)
2. 실행이 끝나면 **성공/실패 티커 리스트를 로그로 저장**
3. 실패 티커만 다시 돌리는 **재시도 루프**를 둔다(2~3회)
4. end는 "오늘"이 아니라 **최신 거래일**로 고정한다

이 4개만 넣으면, 체감 안정성이 확 달라집니다.

---

5) 지금 상황에서의 가장 빠른 복구 절차(추천)

"체크는 하되, 최소로" 하겠습니다. 실제 복구는 아래 순서가 가장 빠릅니다.

1. **Top200으로** `price_update_daily.py` **실행**
2. 실행 후 DB에서 Top200의 max_date가 **동일하게 최신 거래일**인지 1줄 요약 출력
3. 뒤처진 티커만 재시도

---

6) 다음 단계: 원인 확정을 위해 제가 필요한 자료(최소)

정확히 "왜 2025-12-30에서 멈췄는지"를 확정하려면 아래 둘 중 하나만 주시면 됩니다.

- (A) 오늘 실행한 실제 커맨드(그대로 복사)
  예: `python price_update_daily.py ...`
- (B) `price_update_daily.py` 실행 로그(성공/실패 출력)

지금 출력만으로는 **"대부분이 업데이트 대상에 포함되지 않았는지" vs "대상은 포함됐지만 소스가 실패했는지"**를 최종 확정할 수는 없습니다. 다만 확률은 (A) 쪽이 높습니다.

결론

- **DB 자체는 마스터로 쓰기에 충분히 큽니다.** (대부분 장기간 백필 완료)
- 문제는 **데일리 업데이트가 유니버스(Top200/Union) 전체에 적용되지 않는 운영 방식**입니다.

- 해결책은 "키움으로 갈지 여부" 이전에, **tickers-file 기반 실행 + 실패 재시도 + 최신 거래일 고정**을 운영에 박는 것입니다.

원하시면, 지금 업로드된 `price_update_daily.py` 를 기준으로 **(1) 기본값 005930 제거, (2) tickers-file 필수화(경고/종료), (3) 실패 티커 재시도/로그 저장, (4) 최신 거래일 자동 결정**까지 반영한 "운영형 버전"을 바로 만들어 드리겠습니다.

12월 30일에 서버에서 인증이 안되서 업데이트가 안된거야. 그 후론 작업을 하지 않았고. 그러면 어떻게 해야 하지?

상황이 정리됐습니다. **2025-12-30 이후 업데이트가 멈춘 이유는 "서버 인증(또는 접근) 실패"였고, 이후에 업데이트 작업을 재개하지 않아서** DB가 그 날짜에서 멈춰 있는 겁니다. 그러면 할 일은 단순합니다.

핵심 목표는 2가지입니다.

1. **먼저 기존 방식(pykrx 기반)이 다시 정상 동작하는지 확인**
2. 정상이라면 **"2025-12-31 ~ 최신 거래일" 구간을 한 번에 메워서 복구(backfill)하고**, 이후부터는 **Top200만 매일 업데이트**로 운영

아래 순서대로 하시면 됩니다.

---

1) 먼저 "인증 문제가 지금도 남아있는지"를 1분 안에 확인

가장 간단한 재현 테스트입니다. (005930 한 종목만)

powershell 코드 복사

```powershell
cd D:\Quant
.\venv64\Scripts\Activate.ps1
python price_update_daily.py --tickers 005930
```

- 이게 **최신 거래일(오늘 기준 2026-01-26 또는 그 직전 거래일)**까지 들어오면, 인증 문제는 현재 해결된 상태입니다.
- 여전히 실패하면, 먼저 "인증/접근"을 해결해야 합니다(아래 4번 참고).

---

2) 해결된 상태라면: "중단된 구간만" 복구 업데이트(가장 효율적)

이미 DB에는 대부분 종목이 2025-12-30까지 있으니, 마스터 DB를 다시 전부 백필할 필요 없습니다.
2025-12-31부터 최신까지의 '갭'만 메우면 됩니다.

A) Union 619개를 갭 복구(1회)

powershell 코드 복사

```powershell
python price_update_daily.py `
  --tickers-file data\universe_union_kospi_top200_201501_202512.csv `
  --ticker-col ticker `
  --start 20251231
```

중요:

- 지금 `price_update_daily.py` 가 `--start` 인자를 지원하지 않는 구조일 수 있습니다.
  지원하지 않으면 **제가 파일을 바로 수정해서** `--start` **옵션을 넣는 게 최단 경로**입니다.

B) 이후 운영: Top200만 매일 업데이트

powershell 코드 복사

```powershell
python price_update_daily.py `
  --tickers-file data\universe_top200_kospi_20251230.csv `
  --ticker-col ticker
```

(Top200 파일은 가능하면 "오늘 날짜 기준"으로 다시 생성하는 게 이상적입니다.)

---

3) 실패가 다시 발생할 때를 대비한 "운영 안정장치" (반드시 넣어야 합니다)

12/30처럼 인증이 한 번 깨지면 또 같은 일이 생깁니다. 최소한 아래 3개는 넣어야 합니다.

1. **실패 티커 목록 저장**
- 예: `logs/price_update_fail_YYYYMMDD.txt`
2. **재시도(2~3회)**
- 서버/네트워크 이슈는 재시도로 대부분 해결됩니다.
3. **최신 거래일로 end를 고정**

- "오늘"로 조회하면 비거래일/데이터 공백으로 실패하는 경우가 있습니다.

이건 "체크"가 아니라 **장애 복구 자동화**입니다.

---

4) 만약 지금도 "서버 인증"이 안된다면: 선택지는 2개

선택지 A) pykrx 접근 정상화(가장 빠른 임시 복구)

- 네트워크(회사망/방화벽/VPN), SSL/인증서, 라이브러리 버전 문제일 수 있습니다.
- 이 경우 다음을 먼저 시도합니다.
  - `pip install -U pykrx requests certifi`
  - 사내망이면 다른 네트워크(테더링 등)에서 테스트

선택지 B) 키움으로 전환(장기적 안정성 목적)

- 선생님이 이미 준비한 키움 수집기가 있다면,
- "마스터 DB는 키움으로 백필 + 운영도 키움"이 가장 안정적일 수 있습니다.
- 단, 키움은 32bit/로그인/세션 이슈가 있으니 **수집 프로세스 분리**가 필수입니다.

---

5) 제가 지금 바로 도와드릴 수 있는 "최단 해결 루트"

선생님 상황에서는 다음이 가장 현실적입니다.

1. `price_update_daily.py` 에 `--start` **옵션 추가**
2. union 619개에 대해 **start=2025-12-31로 갭 복구 실행**
3. 실행 로그에 **성공/실패 요약 + 실패 티커 파일 저장** 추가
4. 이후 Top200 데일리 운영 스크립트 1개로 고정

진행을 위해 필요한 최소 정보

제가 "수정 파일"을 정확히 만들어 드리려면, `price_update_daily.py` 가 현재 `--start` 를 받는지 여부만 확인하면 됩니다.
가장 빠른 방법은, 아래 명령 출력만 보내주시면 됩니다.

powershell 코드 복사

```powershell
python price_update_daily.py --help
```

이 출력만 있으면, (1) 갭 복구용 옵션 추가 여부, (2) 운영형 안정장치 반영을 바로 확정해서 코드로 드리겠습니다.

'1)번 테스트'는 좀 전에 하지 않았나?

네, 선생님이 **조금 전에 하신 것이 사실상 1)번 테스트를 "부분적으로" 수행하신 것**입니다. 다만 제가 1)번을 다시 언급한 이유는, 현재 상황에서 "성공"을 다음 두 가지로 구분해야 하기 때문입니다.

선생님이 이미 확인하신 것

- 오늘 `price_update_daily.py` 를 실행했을 때 **적어도 1개 종목(보통 005930)이 2026-01-26까지 갱신**됐습니다.
  → 즉, **서버 인증 이슈가 '항상' 막고 있는 상태는 아니다**라는 점은 확인됐습니다.

그런데 왜 "테스트를 다시"라고 한 것처럼 보였나

지금 DB 결과를 보면,

- **621개 중 570개가 2025-12-30에서 그대로 멈춰 있고**
- 최신(2026-01-26)은 **1개 종목에만 존재**합니다.

이 패턴이면, 원인 후보가 두 가지로 갈립니다.

1. **업데이트 대상이 005930만이었다**(인자 없이 실행/기본값)
2. **여러 종목을 업데이트하려 했지만, 12/30 이후 구간에서 대부분 실패했다**(인증/접근 문제 재발)

이 둘은 대응이 완전히 다릅니다.

- (1)이면: 그냥 `--tickers-file` 로 619/200을 넣어 돌리면 됩니다.
- (2)면: 인증/접근/재시도/최신거래일 고정 같은 "운영 안정화"를 먼저 해야 합니다.

그래서 제가 1)번 테스트를 "명확하게" 다시 하자고 제안한 겁니다.
선생님이 이미 "005930 단일 종목 최신 갱신"을 확인하셨다면, **지금 필요한 다음 확인은 이것 하나입니다.**

지금 필요한 '단 한 번'의 확인

Top200(또는 union 619) 파일을 넣어서 실제로 다수 종목이 2025-12-31 이후 구간을 가져오는지를 확인해야 합니다.

예: (Top200으로 먼저 테스트 권장)

powershell 코드 복사

```powershell
python price_update_daily.py `
  --tickers-file data\universe_top200_kospi_20251230.csv `
  --ticker-col ticker
```

- 이걸 돌렸는데도 여전히 대부분이 2025-12-30에서 멈추면 → 인증/접근 문제가 다수 호출에서 재발하는 겁니다.
- 반대로 정상적으로 2026-01-26 근처까지 다수 종목이 갱신되면 → 이전엔 단지 "005930만 돌린 것"에 가깝습니다.

요약하면, **네, 단일 종목(005930) 기준 테스트는 사실상 이미 하셨습니다.**
이제는 "다수 종목(tickers-file)" 기준으로 한 번만 실행해 보면 원인이 즉시 갈립니다.